ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Lê Minh

PHÁT HIỆN LỖ HỔNG PHÂN QUYỀN CÓ TRẠNG THÁI TRONG REST API

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN LÊ MINH

PHÁT HIỆN LỖ HỔNG PHÂN QUYỀN CÓ TRẠNG THÁI TRONG REST API

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: ThS. Trần Mạnh Cường

Cán bộ đồng hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Nguyen Le Minh

DETECTING STATEFUL AUTHORIZATION VULNERABILITIES IN REST APIs

BACHELOR'S THESIS

Major: Information Technology

Supervisor: MSc. Tran Manh Cuong

Co-Supervisor: Dr. Nguyen Thi Thuy

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất tới ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy, những người đã tận tình chỉ dẫn, định hướng khoa học và động viên tôi trong suốt quá trình nghiên cứu và hoàn thành khóa luận này.

Tôi xin chân thành cảm ơn các thầy cô giáo tại Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội. Các thầy cô đã truyền đạt kiến thức nền tảng quý báu trong suốt những năm tháng học tập tại trường.

Cuối cùng, tôi xin bày tỏ lòng biết ơn sâu sắc tới gia đình và người thân. Gia đình đã luôn ủng hộ, chia sẻ và tạo mọi điều kiện để tôi hoàn thành khóa luận tốt nghiệp.

Hà Nội, tháng 11 năm 2026

Nguyễn Lê Minh

Lời cam đoan

Tôi là Nguyễn Lê Minh, sinh viên lớp K68I-IT3, mã sinh viên 23020114. Tôi xin cam đoan khóa luận tốt nghiệp này là công trình nghiên cứu do chính tôi thực hiện dưới sự hướng dẫn của ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy. Mọi kết quả thực nghiệm, số liệu đo đạc và mã nguồn trình bày trong khóa luận đều trung thực, được thu thập từ các lượt chạy thực tế. Công trình không sao chép từ bất kỳ tài liệu nào khác mà không được trích dẫn rõ ràng.

Hà Nội, ngày ..... tháng ..... năm 2026

Sinh viên

Nguyễn Lê Minh

Tóm tắt

Kiểm soát truy cập giới hạn những thao tác mà mỗi người dùng được phép thực hiện trên REST API. Các kỹ thuật kiểm thử tự động hiện nay chủ yếu tập trung vào lỗi phân quyền phi trạng thái, như lỗi ở mức đối tượng (BOLA) hoặc mức chức năng (BFLA). Lớp lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control) ít được các kỹ thuật này kiểm tra trực tiếp. Lỗi này xuất hiện khi quyền hạn của người dùng bị thu hồi trong vận hành, nhưng hệ thống vẫn tiếp tục chấp nhận các phiên làm việc cũ.

Khóa luận đề xuất giải pháp tự động phát hiện lớp lỗ hổng này bằng kiểm thử suy biến trên công cụ EvoMaster. Giải pháp dựa trên quan hệ MR-A1: thao tác đã thành công trước khi tài khoản bị hạ quyền phải bị từ chối khi gửi lại cùng token sau đó. Mỗi kịch bản kiểm thử gồm bộ ba thao tác `[source/transition/follow-up]`. Mô hình Naive Bayes tự động nhận diện thao tác hạ quyền từ đặc tả OpenAPI, nhờ đó giảm cấu hình thủ công.

Thực nghiệm gồm 98 lượt chạy ở chế độ hộp đen trên bốn ứng dụng mục tiêu. reports-api và vampi-seeded mang lỗi gieo; pyLoad mang lỗ hổng CVE-2026-41133; artifact-keeper mang một lỗi mà nhà phát triển đã vá. Trên các biến thể có lỗ hổng, công cụ phát hiện vi phạm ở 19/20 lượt với reports-api, 7/20 lượt với vampi-seeded, 5/5 lượt với pyLoad và 5/5 lượt với artifact-keeper. Trên 48 lượt chạy với các biến thể đã vá, gồm bản vá của chính nhà phát triển artifact-keeper, công cụ không phát sinh báo động giả nào. Quan hệ MR-A1 phát hiện được việc máy chủ tiếp tục chấp nhận quyền đã thu hồi trên cả bốn ứng dụng đã khảo sát.

Từ khóa: REST API, kiểm soát truy cập, phân quyền có trạng thái, kiểm thử suy biến, EvoMaster

Abstract

Access control restricts the operations that each user is allowed to perform on a REST API. Current automated testing techniques mainly target stateless authorization flaws, such as object-level (BOLA) or function-level (BFLA) flaws. Stateful broken access control is rarely checked directly by these techniques. This flaw occurs when a user's privileges are revoked at runtime, but the system keeps accepting the user's existing sessions.

This thesis proposes an automated approach to detect this class of vulnerabilities through metamorphic testing in the EvoMaster tool. The approach is based on the metamorphic relation MR-A1: an operation that succeeded before the account was downgraded must be rejected when it is sent again later with the same token. Each test scenario consists of an operation triple `[source/transition/follow-up]`. A Naive Bayes model automatically identifies downgrade operations from the OpenAPI specification, which reduces manual configuration.

The experiments comprise 98 black-box runs on four target applications. Two of them, reports-api and vampi-seeded, carry seeded bugs. pyLoad carries the vulnerability CVE-2026-41133, and artifact-keeper carries a bug that its developers have since fixed. On the vulnerable variants, the tool detected violations in 19/20 runs on reports-api, 7/20 runs on vampi-seeded, 5/5 runs on pyLoad and 5/5 runs on artifact-keeper. Across 48 runs on the fixed variants, including the developers' own fix of artifact-keeper, the tool raised no false alarms. MR-A1 detected servers that keep honouring revoked privileges in all four studied applications.

Keywords: REST API, access control, stateful authorization, metamorphic testing, EvoMaster

Mục lục

Lời cảm ơn

Lời cam đoan **i**

Tóm tắt **ii**

Abstract **iii**

Mục lục **iv**

Danh sách hình vẽ **vi**

Danh sách bảng **vii**

Danh mục các từ viết tắt **viii**

Chương 1 Giới thiệu **1**

1.1 Bối cảnh và động lực nghiên cứu 1

1.2 Phát biểu bài toán 1

1.3 Mục tiêu và phạm vi nghiên cứu 2

1.4 Đóng góp của khóa luận 3

1.5 Bố cục khóa luận 3

Chương 2 Cơ sở lý thuyết và công việc liên quan **5**

2.1 Kiểm thử REST API dựa trên tìm kiếm 5

2.2 Kiểm thử suy biến 5

2.3 Lỗ hổng kiểm soát truy cập trong REST API 6

2.4 Các công trình nghiên cứu liên quan 7

Chương 3 Phương pháp đề xuất **9**

3.1 Tổng quan giải pháp và kiến trúc Oracle 970 9

3.2 Quan hệ kiểm thử suy biến MR-A1 11

3.3 Bộ ba thao tác và điều kiện tiền đề 12
3.3.1 Cấu trúc bộ ba thao tác 14
3.3.2 Điều kiện tiền đề và giả định trên tài nguyên 14
3.3.3 Bộ ba bị hủy và bước đăng nhập lại 15
3.3.4 Thuật toán dựng kịch bản kiểm thử MR-A1 16
3.4 Tự động nhận diện và xếp hạng thao tác chuyển trạng thái 16
3.4.1 Tiếp cận dựa trên bộ từ vựng 18
3.4.2 Tiếp cận dựa trên mô hình Naive Bayes 19

Chương 4 Cài đặt thực nghiệm và tích hợp hệ thống 21
4.1 Tích hợp Oracle 970 vào mã nguồn EvoMaster 21
4.2 Xây dựng tập benchmark và ứng dụng mục tiêu 22
4.2.1 Ứng dụng reports-api 22
4.2.2 Ứng dụng vampi-seeded 23
4.2.3 Ứng dụng pyLoad 23
4.2.4 Ứng dụng artifact-keeper 24
4.3 Thiết lập thực nghiệm 25

Chương 5 Thực nghiệm và đánh giá kết quả 26
5.1 Thiết kế thực nghiệm 26
5.2 Hiệu quả phát hiện và độ chính xác 26
5.3 Hiệu quả xếp hạng thao tác chuyển trạng thái 28
5.4 Nghiên cứu tình huống trên lỗi thật 30
5.5 Phân tích nguyên nhân bỏ sót và giới hạn 30
5.6 Đối chứng với baseline và kiểm tra upstream 32

Chương 6 Kết luận và hướng phát triển 34
6.1 Tổng kết kết quả nghiên cứu 34
6.2 Hạn chế hiện tại của đề tài 35
6.3 Hướng phát triển trong tương lai 35

Tài liệu tham khảo 36

Phụ lục A Danh mục endpoint của các ứng dụng mục tiêu 39

Danh sách hình vẽ

1.1 Vòng đời lỗ hổng phân quyền có trạng thái 2

2.1 Chu trình kiểm thử EvoMaster với pha kiểm thử bảo mật 6

3.1 Trình tự thực hiện bộ ba thao tác và kích hoạt Oracle 970 10
3.2 Không gian trạng thái phân quyền và quan hệ MR-A1 13
3.3 Quy trình nhận diện thao tác hạ quyền từ đặc tả OpenAPI 18

4.1 Ba thành phần phần mở rộng trong chu trình EvoMaster 22

Danh sách bảng

3.1 Ánh xạ giá trị quan sát sang mã trạng thái HTTP 11
3.2 Đối chiếu oracle so sánh chéo tài khoản và quan hệ MR-A1 12

4.1 Các ứng dụng mục tiêu 23
4.2 Thiết lập chạy theo ứng dụng 25

5.1 Kết quả phát hiện và báo động giả theo lượt chạy 27
5.2 Phân loại kết quả theo lượt chạy 27
5.3 Kết quả xếp hạng thao tác đích trên 47 bản đặc tả 29
5.4 Các lớp lỗi EvoMaster v6.2.0 phát sinh trên biến thể có lỗ hổng . . . 32

A.1 Các endpoint trong bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu 39

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
API	Application Programming Interface	Giao diện lập trình ứng dụng
artifact-keeper	–	Kho artifact mã nguồn mở, ứng dụng mục tiêu mang lỗi thật
BFLA	Broken Function Level Authorization	Lỗi phân quyền cấp chức năng
BOLA	Broken Object Level Authorization	Lỗi phân quyền cấp đối tượng
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CVE	Common Vulnerabilities and Exposures	Mã định danh lỗ hổng
CVSS	Common Vulnerability Scoring System	Thang điểm lỗ hổng
CWE	Common Weakness Enumeration	Danh mục điểm yếu phần mềm
FN	False Negative	Bỏ sót
FP	False Positive	Báo động giả
GHSA	GitHub Security Advisories	Cảnh báo bảo mật của GitHub
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
INCITS	International Committee for Information Technology Standards	Ủy ban quốc tế về tiêu chuẩn công nghệ thông tin
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu
JWT	JSON Web Token	Chuẩn token dạng JSON dùng cho xác thực
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MIO	Many Independent Objective	Thuật toán nhiều mục tiêu độc lập
MR	Metamorphic Relation	Quan hệ suy biến
MR-A1	Metamorphic Relation A1	Quan hệ suy biến: phiên cũ phải bị từ chối sau khi hạ quyền
Oracle	Test Oracle	Cơ chế phán quyết kết quả ca kiểm thử là đúng hay có lỗi

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
OWASP	Open Worldwide Application Security Project	Dự án mở toàn cầu về an toàn ứng dụng
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
RFC	Request for Comments	Loạt tài liệu đặc tả chuẩn Internet
REST	Representational State Transfer	Kiểu kiến trúc dịch vụ web
SCIM	System for Cross-domain Identity Management	Chuẩn quản lý định danh liên miền
SMRL	Security Metamorphic Relation Language	Ngôn ngữ đặc tả quan hệ suy biến an toàn
TN	True Negative	Không báo đúng
TP	True Positive	Phát hiện đúng

Chương 1

Giới thiệu

Khi quyền của người dùng thay đổi, máy chủ có thể vẫn chấp nhận thông tin xác thực đã cấp trước đó cho thao tác đã mất quyền. Chương này giới thiệu bài toán phát hiện lỗ hổng này, cùng mục tiêu, phạm vi và đóng góp của khóa luận.

1.1 Bối cảnh và động lực nghiên cứu

OWASP API Security Top 10 năm 2023 xếp lỗ hổng phân quyền ở mức đối tượng và mức chức năng vào nhóm rủi ro hàng đầu [1]. Khi hệ thống yêu cầu thu hồi quyền có hiệu lực ngay, nguy cơ an ninh phát sinh nếu phiên cũ vẫn giữ được quyền này. Hiện tượng máy chủ tiếp tục chấp nhận thao tác theo quyền đã thu hồi được gọi là lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control).

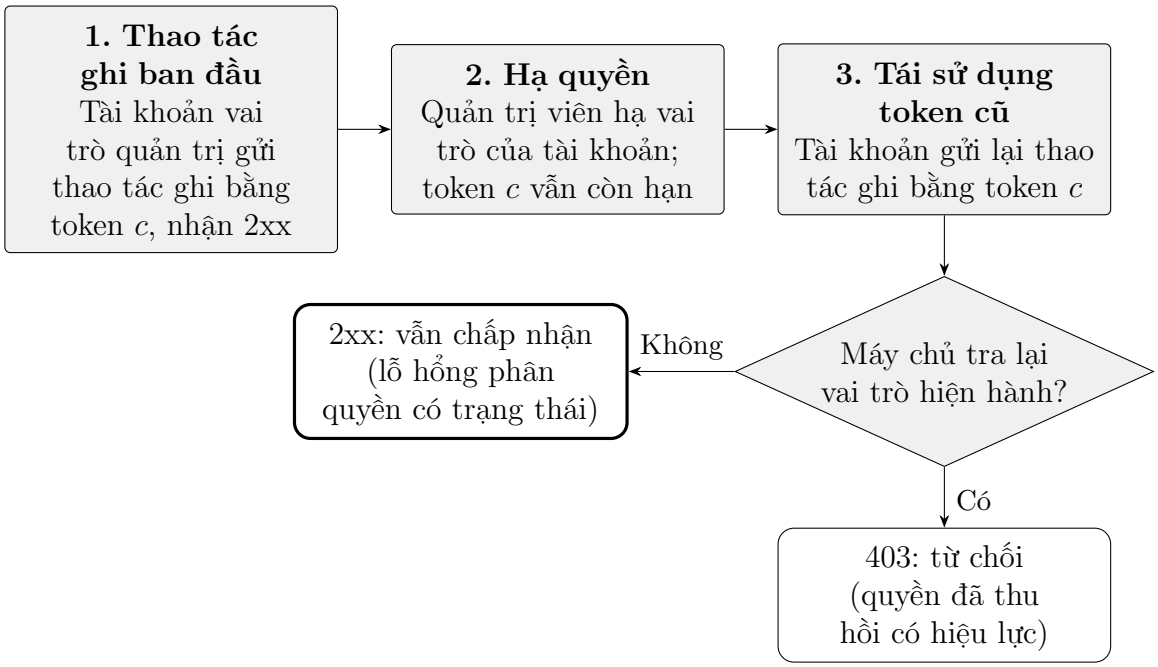
EvoMaster áp dụng kiểm thử dựa trên tìm kiếm để tự động sinh ca kiểm thử cho REST API [2]. Với lỗi phân quyền, mã trạng thái của một yêu cầu HTTP thường không đủ để biết kết quả có đúng chính sách truy cập hay không. Các oracle phân quyền hiện có trong EvoMaster đối chiếu hành vi giữa các tài khoản có quyền khác nhau [3]. Khóa luận tập trung vào trường hợp quyền của một tài khoản thay đổi, nhưng thông tin xác thực cấp trước đó vẫn được chấp nhận cho thao tác đã mất quyền.

1.2 Phát biểu bài toán

Trong phạm vi khóa luận, lỗ hổng phân quyền có trạng thái xảy ra khi hệ thống tiếp tục chấp nhận một thao tác phụ thuộc vào quyền đã thu hồi. Thao tác đó được gửi bằng thông tin xác thực cấp trước lần thay đổi quyền. Chẳng hạn, sau khi một tài khoản bị hạ xuống vai trò thấp hơn, máy chủ vẫn chấp nhận token hoặc cookie cấp trước đó cho thao tác không còn được phép. Hiện tượng này có thể thuộc điểm yếu quản lý hết hiệu lực phiên CWE-613, tùy chính sách thu hồi quyền và cơ chế xác thực của hệ thống [4]. Hình 1.1 minh họa ba giai đoạn của lỗ

hổng: thao tác ghi ban đầu, thao tác hạ quyền và tái sử dụng token cũ.

Khóa luận tập trung tự động phát hiện việc máy chủ tiếp tục chấp nhận quyền đã thu hồi. Hành vi này không thể phán quyết chỉ từ một yêu cầu HTTP riêng lẻ. Phán quyết cần kết quả của cùng thao tác trước và sau khi thay đổi quyền, cùng dấu hiệu cho thấy quyền đã thật sự bị thu hồi.



Hình 1.1: Vòng đời lỗ hổng phân quyền có trạng thái

1.3 Mục tiêu và phạm vi nghiên cứu

Xuất phát từ bài toán trên, khóa luận đặt mục tiêu mở rộng EvoMaster để tự động phát hiện các vi phạm phân quyền có trạng thái thuộc quan hệ suy biến MR-A1. Phần mở rộng chạy trong pha kiểm thử bảo mật, tiếp sau quá trình tìm kiếm tiến hóa, nên thuật toán tìm kiếm Many Independent Objective (MIO) của công cụ được giữ nguyên.

Về phạm vi, khóa luận chỉ xét quan hệ MR-A1, trong đó quyền thực hiện một thao tác được kiểm tra trước và sau khi vai trò của chủ thể bị hạ. Kịch bản kiểm thử MR-A1 được dựng từ đặc tả OpenAPI, cấu hình xác thực, các quan sát HTTP và kho lưu trữ ca kiểm thử của EvoMaster. Phương pháp chỉ áp dụng khi có đủ ba điều kiện: thao tác hạ quyền thực hiện được qua REST API, hệ thống cho phép đăng nhập lại để lấy thông tin xác thực mới sau khi hạ quyền, và hệ thống từ chối

thao tác không đủ quyền bằng mã 403. Các tình huống không thỏa các điều kiện này nằm ngoài phạm vi đánh giá. Chẳng hạn, thao tác xóa chủ thể không để lại tài khoản nào để đăng nhập lại.

1.4 Đóng góp của khóa luận

Khóa luận có bốn đóng góp chính:

1. **Quan hệ suy biến MR-A1.** Định nghĩa quan hệ MR-A1 để đối chiếu hành vi phân quyền trước và sau thao tác hạ quyền, với thông tin xác thực giữ cố định. Quan hệ được cài đặt bằng bộ ba thao tác `[source/transition/follow-up]` và điều kiện tiền đề.

2. **Tự động nhận diện thao tác chuyển trạng thái.** Xây dựng và đánh giá hai cách xếp hạng thao tác hạ quyền từ đặc tả OpenAPI: bộ từ vựng có trọng số và mô hình Naive Bayes.

3. **Kiểm chứng bằng bước đăng nhập lại.** Tài khoản nạn nhân lấy token mới và chạy lại chuỗi lời gọi của mình; bộ ba bị loại nếu token mới không bị từ chối.

4. **Đánh giá thực nghiệm.** Đánh giá Oracle 970 trên bốn ứng dụng. Hai ứng dụng reports-api và vampi-seeded mang lỗi gieo. Hai ứng dụng mã nguồn mở mang lỗi thật: pyLoad với lỗ hổng CVE-2026-41133, và artifact-keeper với một lỗi đã được nhà phát triển vá. Trên các biến thể có lỗ hổng, Oracle 970 phát hiện vi phạm ở 36/50 lượt, gồm 5/5 lượt trên pyLoad và 5/5 lượt trên artifact-keeper. Trong 48 lượt chạy trên biến thể đã vá, 30 lượt dựng được bộ ba và Oracle 970 không báo động giả ở lượt nào.

1.5 Bố cục khóa luận

Sau Chương 2 về cơ sở lý thuyết và các công trình liên quan, Chương 3 định nghĩa quan hệ MR-A1, cách dựng kịch bản kiểm thử và cách nhận diện thao tác hạ quyền. Chương 4 trình bày cách tích hợp Oracle 970 vào EvoMaster, cùng các ứng dụng mục tiêu và thiết lập thực nghiệm. Trên cơ sở đó, Chương 5 đánh giá khả

năng phát hiện vi phạm, khả năng xếp hạng thao tác chuyển trạng thái và các giới hạn quan sát được. Chương 6 tổng kết kết quả và đề xuất hướng phát triển.

Chương 2

Cơ sở lý thuyết và công việc liên quan

Phát hiện vi phạm phân quyền sau khi quyền thay đổi đòi hỏi kết hợp ba nền tảng: kiểm thử REST API dựa trên tìm kiếm, kiểm thử suy biến và kiểm soát truy cập. Chương này trình bày ba nền tảng đó và đối chiếu các công trình liên quan để xác định khoảng trống của khóa luận.

2.1 Kiểm thử REST API dựa trên tìm kiếm

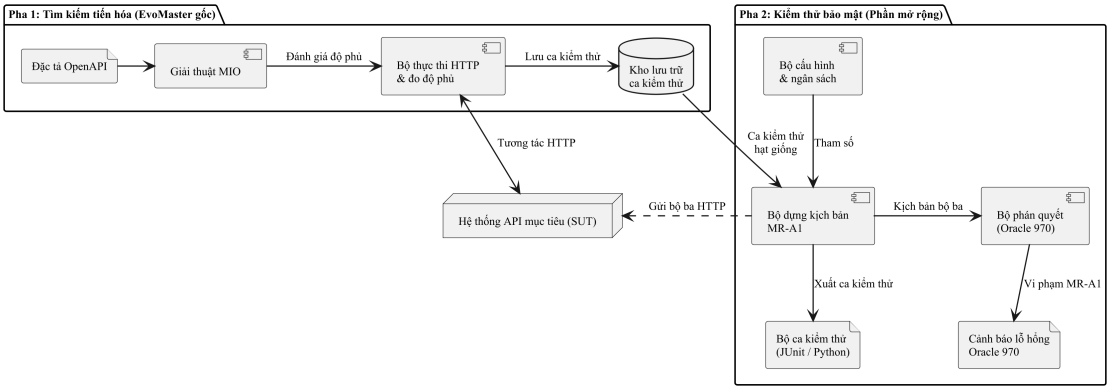
Kiểm thử phần mềm dựa trên tìm kiếm (search-based software testing) coi việc sinh ca kiểm thử là một bài toán tối ưu hóa. Hàm độ thích nghi (fitness function) đo mức độ một cá thể kiểm thử đáp ứng mục tiêu tìm kiếm, chẳng hạn khoảng cách đến một nhánh chương trình. Thuật toán tìm kiếm dùng giá trị này để chọn và biến đổi cá thể kiểm thử [5]. Với REST API, thuật toán sinh và biến đổi các chuỗi yêu cầu HTTP dựa trên phản hồi thu được khi thực thi.

EvoMaster là công cụ sinh ca kiểm thử tự động cho REST API theo hướng tìm kiếm [2, 6]. Công cụ dùng thuật toán Many Independent Objective (MIO), trong đó các cá thể được quản lý và đánh giá theo từng mục tiêu [7]. EvoMaster hỗ trợ chế độ hộp đen chỉ dùng đặc tả OpenAPI và chế độ hộp trắng có đo độ phủ mã [8, 9]. Kho lưu trữ ca kiểm thử giữ các cá thể đại diện cho những mục tiêu đã đạt, phục vụ việc tổng hợp bộ ca kiểm thử và các pha tiếp theo.

Pha kiểm thử bảo mật của EvoMaster chạy sau pha tìm kiếm tiến hóa [3]. Pha này dùng các cá thể và kết quả thực thi trong kho lưu trữ để dựng thêm kịch bản kiểm thử thuộc tính bảo mật, không cần thay đổi thuật toán MIO. Hình 2.1 biểu diễn chu trình kiểm thử và vị trí pha kiểm thử bảo mật.

2.2 Kiểm thử suy biến

Oracle kiểm thử (test oracle) là cơ chế đối chiếu kết quả thực thi với hành vi kỳ vọng hoặc một tiêu chí kiểm tra xác định trước [10]. Khi không thể xây dựng một



Hình 2.1: Chu trình kiểm thử EvoMaster với pha kiểm thử bảo mật

oracle đầy đủ hoặc chi phí xây dựng quá lớn, quá trình kiểm thử gặp bài toán thiếu oracle (test oracle problem). Kiểm thử suy biến (metamorphic testing) hỗ trợ kiểm thử khi thiếu oracle bằng cách đối chiếu quan hệ kỳ vọng giữa những lần thực thi có liên quan [11]. Một quan hệ suy biến (metamorphic relation) phát biểu mối liên hệ kỳ vọng giữa đầu vào và đầu ra của các lần thực thi đó. Khi các quan sát thỏa điều kiện áp dụng nhưng vi phạm quan hệ, kết quả là bằng chứng về hành vi không phù hợp với thuộc tính được kiểm tra [12].

Ngoài kiểm thử chức năng, kiểm thử suy biến cũng đã được áp dụng cho bài toán an toàn bảo mật. Chẳng hạn, SMRL (Security Metamorphic Relation Language) là ngôn ngữ đặc tả biểu diễn các thuộc tính bảo mật của ứng dụng web dưới dạng quan hệ suy biến [13]. Công trình này gần với khóa luận về phương pháp luận, nhưng không tích hợp một oracle thu hồi quyền có trạng thái vào chu trình fuzzing REST API của EvoMaster.

2.3 Lỗ hổng kiểm soát truy cập trong REST API

Kiểm soát truy cập (access control) là cơ chế xác định một chủ thể được phép thực hiện những thao tác nào trên từng tài nguyên của hệ thống. Lỗ hổng phân quyền ở mức đối tượng (Broken Object Level Authorization, BOLA) phát sinh khi API không kiểm tra đủ quyền của chủ thể đối với đối tượng được yêu cầu. Lỗ hổng phân quyền ở mức chức năng (Broken Function Level Authorization, BFLA) phát sinh khi API không kiểm tra đủ quyền gọi một chức năng. Cả hai lớp lỗi nằm trong OWASP API Security Top 10 [1].

Các oracle BOLA và BFLA hiện có đối chiếu phản hồi của các tài khoản có quyền khác nhau. Khác với các oracle này, khóa luận xét trường hợp cùng một chủ thể vẫn được chấp nhận sau khi quyền cần thiết đã bị thu hồi. Hiện tượng này được gọi là lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control). Với hệ thống có chính sách thu hồi quyền có hiệu lực ngay, các yêu cầu tiếp theo phụ thuộc vào quyền vừa bị thu hồi phải bị từ chối. Chuẩn kiểm soát truy cập theo vai trò RBAC mô tả các lệnh thu hồi như hủy gán người dùng khỏi vai trò và thu hồi quyền khỏi vai trò [14, 15].

Các mô hình RBAC mô tả quan hệ giữa vai trò, quyền và các thao tác quản lý. Hiệu lực của việc thay đổi các quan hệ này đối với phiên đang hoạt động phụ thuộc vào chính sách và cơ chế thực thi của hệ thống. Việc tiếp tục chấp nhận thông tin xác thực cũ sau khi thu hồi quyền có thể thuộc CWE-613 khi chính sách yêu cầu chấm dứt hoặc cập nhật phiên [4]. Chuẩn quản lý danh tính SCIM (System for Cross-domain Identity Management, RFC 7643) định nghĩa các trường trạng thái kích hoạt, vai trò và phân quyền trong lược đồ người dùng. Ý nghĩa phân quyền cụ thể của các trường này do nhà cung cấp dịch vụ quy định [16].

2.4 Các công trình nghiên cứu liên quan

Các công cụ sinh ca kiểm thử REST API như RESTler và RestTestGen chủ yếu hướng tới độ phủ hành vi và lỗi chức năng. RESTler sinh các chuỗi lời gọi có trạng thái kèm một số bộ kiểm tra lỗi [17]. RestTestGen khai thác quan hệ phụ thuộc giữa các thao tác để dựng chuỗi kiểm thử [18].

Các nghiên cứu gần đây về kiểm thử phân quyền tập trung vào lỗi ở mức đối tượng và mức chức năng. Şahin, Zhang và Arcuri bổ sung vào EvoMaster một nhóm oracle an toàn dưới dạng pha hậu xử lý, so sánh nhiều tài khoản tại cùng một thời điểm [3, 19]. BACScan dùng bằng chứng từ phản hồi nghiệp vụ để phát hiện thao tác truy cập hoặc sửa đổi trái phép trong ứng dụng web [20]. GraphQLer khai thác ngữ cảnh và chuỗi thao tác khi kiểm thử API GraphQL [21].

Hai công trình khác tiếp cận bài toán bằng kiểm thử suy biến và fuzzing kiểm soát truy cập. ARMeta dùng quy trình đa tác tử dựa trên mô hình ngôn ngữ lớn để xây dựng kịch bản kiểm thử suy biến cho REST API [22]. BACFuzz là fuzzer kiểm soát truy cập dạng hộp xám cho PHP, với oracle ở mức truy vấn cơ sở dữ liệu [23].

Trong các công trình nêu trên, chưa có oracle nào thuộc một fuzzer REST API kết hợp đồng thời ba cơ chế của MR-A1. Ba cơ chế đó là tự động dựng thao tác thu hồi quyền, dùng lại token cấp trước khi thu hồi, và phán quyết dựa trên quan sát trước và sau khi thay đổi quyền. Phạm vi khảo sát gồm các công trình được trích dẫn và mã nguồn upstream của EvoMaster (bản do nhóm tác giả gốc duy trì) tại thời điểm kiểm tra năm 2026; Mục 5.6 kiểm thêm bằng cách chạy upstream trên các ứng dụng mục tiêu. Trên cơ sở đối chiếu này, Chương 3 định nghĩa quan hệ MR-A1 và phương pháp cài đặt quan hệ này trong EvoMaster.

Chương 3

Phương pháp đề xuất

Phương pháp đề xuất phát hiện lỗ hổng phân quyền có trạng thái bằng cách đối chiếu kết quả của cùng một thao tác trước và sau khi quyền của chủ thể bị thu hồi. Chương này trình bày phương pháp từ quan hệ suy biến MR-A1 đến kịch bản kiểm thử chạy được trong EvoMaster.

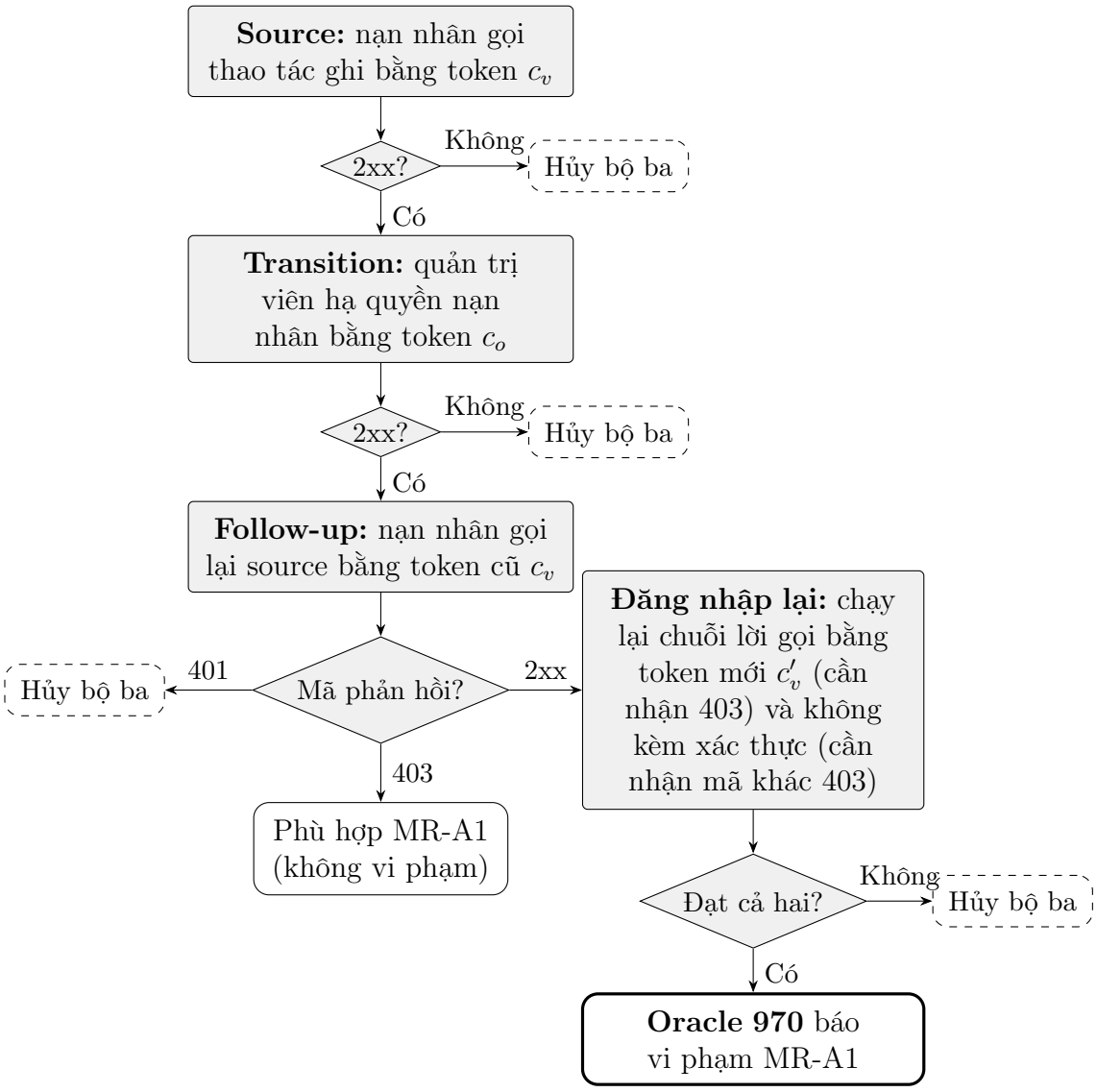
3.1 Tổng quan giải pháp và kiến trúc Oracle 970

Với kiểm thử phân quyền REST API, bài toán thiếu oracle (Mục 2.2) phát sinh vì mã trạng thái HTTP không phản ánh đúng quyết định phân quyền. Một yêu cầu có thể nhận mã `200 OK` ngay cả khi máy chủ chấp nhận thao tác trái với chính sách phân quyền.

Oracle 970 thực hiện phép so sánh theo thời gian trong pha kiểm thử bảo mật của EvoMaster v6.1.1, theo kiến trúc oracle an toàn của Arcuri và Şahin [3, 19]. Mã 970 là mã lỗi mà phần mở rộng gán cho vi phạm MR-A1 trong nhóm oracle thử nghiệm của EvoMaster. Pha kiểm thử bảo mật (Mục 2.1) lấy đầu vào từ kho lưu trữ ca kiểm thử [2, 3].

Phương pháp tách hai thành phần: bộ dựng kịch bản kiểm thử tạo các bộ ba thao tác `[source/transition/follow-up]`, còn bộ phán quyết đánh giá các bộ ba đã thực hiện thành công. Cách tách này phân biệt được lượt không dựng được bộ ba hợp lệ với lượt có bộ ba nhưng không có vi phạm. Hình 3.1 trình bày trình tự thực hiện bộ ba và điều kiện kích hoạt Oracle 970.

Nhóm oracle so sánh chéo tài khoản (differential oracle) của Şahin và cộng sự đối chiếu quyền giữa nhiều tài khoản tại cùng thời điểm để phát hiện lỗi BOLA và BFLA [3]. Ngược lại, Oracle 970 theo dõi cùng một tài khoản trước và sau khi quyền thay đổi, với thông tin xác thực giữ nguyên.



Hình 3.1: Trình tự thực hiện bộ ba thao tác và kích hoạt Oracle 970

3.2 Quan hệ kiểm thử suy biến MR-A1

Dựa trên nguyên lý kiểm thử suy biến ở Mục 2.2, MR-A1 xác định quan hệ kỳ vọng giữa hai lần thực hiện một thao tác, trước và sau khi thu hồi quyền [11, 12].

MR-A1 xét một chủ thể p, tức tài khoản được kiểm tra, có quyền thực hiện thao tác trước khi bị hạ quyền. Chủ thể này gửi yêu cầu bằng thông tin xác thực c, có thể là token hoặc cookie phiên định danh p. Thao tác a là một lời gọi REST API ghi dữ liệu có kiểm soát phân quyền, chẳng hạn cập nhật một báo cáo, việc chỉ quản trị viên được làm. Trạng thái s mô tả phân quyền của hệ thống tại thời điểm quan sát. Thao tác chuyển trạng thái T (transition) do một tài khoản quản trị thực hiện, nhằm hạ vai trò hoặc thu hồi quyền của p.

Hàm quan sát $\mathrm{Obs}(a, c, s)$ lấy kết quả từ mã trạng thái HTTP khi chủ thể dùng c gọi thao tác a tại trạng thái s [24]. Bảng 3.1 nêu ba giá trị quan sát và mã trạng thái tương ứng. Mã `2xx` cho biết máy chủ chấp nhận yêu cầu ở tầng HTTP, nhưng không xác nhận trạng thái dữ liệu nghiệp vụ sau thao tác. Oracle phán quyết dựa trên mã 403. Mã `401` ở bước follow-up thường gặp ở hệ thống thu hồi quyền bằng cách vô hiệu hóa phiên, chẳng hạn bản vá của artifact-keeper (Mục 5.4), và nằm ngoài phạm vi phán quyết. Mã 404 không được coi là lời từ chối.

Bảng 3.1: Ánh xạ giá trị quan sát sang mã trạng thái HTTP

Giá trị	Mã HTTP	Ý nghĩa
ALLOWED	2xx	Máy chủ chấp nhận lời gọi ở tầng HTTP.
DENIED	403	Máy chủ từ chối lời gọi trên cơ sở phân quyền.
INCONCLUSIVE	401	Phiên hết hạn hoặc chưa xác định được quyền; bộ ba bị hủy và dựng lại.

Gọi $s' = T(s)$ là trạng thái sau khi thực hiện T. Khi c được giữ cố định qua hai lần quan sát, MR-A1 có dạng Công thức (3.1).

$$\mathrm{Obs}(a, c, s) = \textsc{allowed} \ \wedge \ \mathit{Pre}(a, T) \implies \mathrm{Obs}(a, c, s') = \textsc{denied}. \tag{3.1}$$

Trong đó, $\mathit{Pre}(a, T)$ là điều kiện tiền đề, yêu cầu thao tác a cần đúng quyền mà T thu hồi đối với chủ thể đang xét. Khi vế trái đúng, kết quả kỳ vọng ở trạng thái s' là DENIED. Nếu kết quả vẫn là ALLOWED, MR-A1 bị vi phạm và oracle báo lỗi, vì máy chủ vẫn chấp nhận thông tin xác thực cũ cho thao tác đã mất quyền.

Theo giả định của phương pháp, thao tác follow-up giữ nguyên token ban đầu mà không đăng nhập lại để tái hiện lỗi phiên cũ vẫn giữ quyền đã thu hồi. Hiện tượng này có thể thuộc điểm yếu CWE-613 [4] (Mục 1.2); các oracle sẵn có của EvoMaster chưa kiểm tra nó.

Điều kiện tiền đề loại các thao tác không phụ thuộc vào quyền bị thu hồi, nhờ đó giảm báo động giả. Chẳng hạn, thao tác cập nhật hồ sơ cá nhân dùng chung cho nhiều vai trò, nên việc máy chủ tiếp tục chấp nhận nó sau khi hạ quyền là hành vi đúng. Chuẩn RBAC để ngỏ cách cập nhật quyền trên phiên đang hoạt động [14], nên yêu cầu từ chối ngay sau khi hạ quyền là chính sách riêng của từng hệ thống mục tiêu.

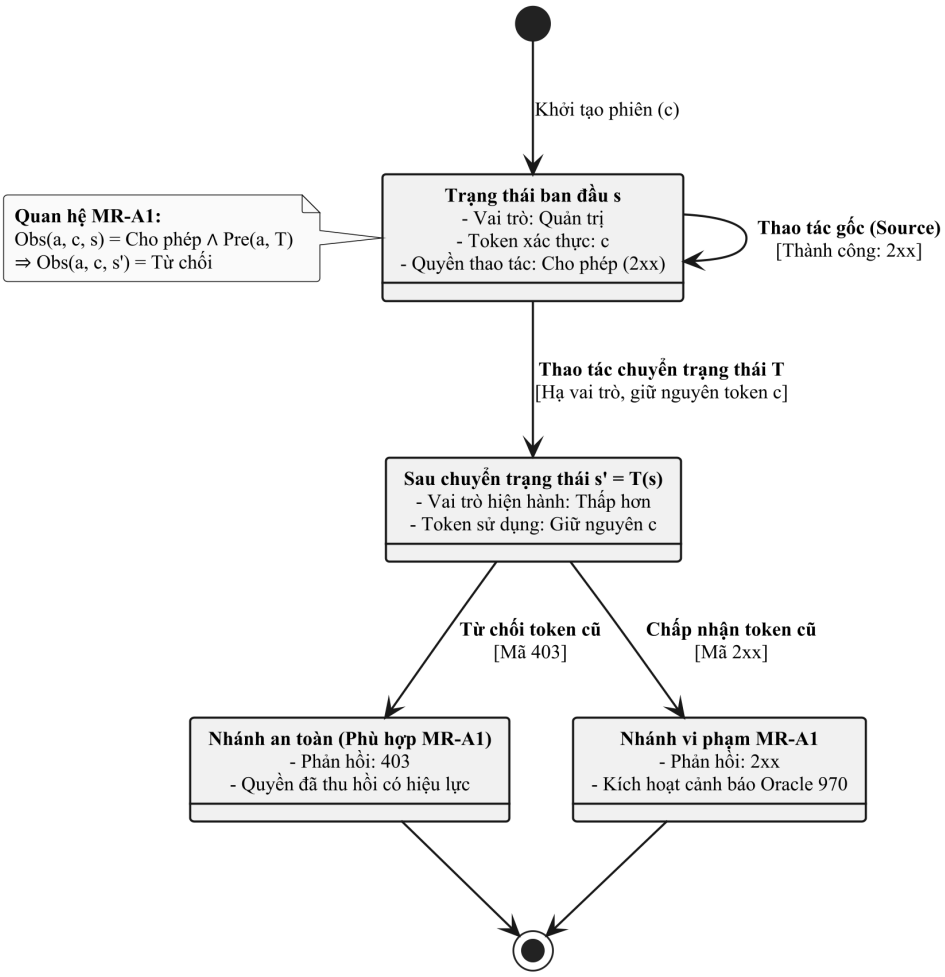
Bảng 3.2 đối chiếu MR-A1 với các oracle so sánh chéo tài khoản đã nêu ở Mục 3.1. Hình 3.2 biểu diễn không gian chuyển trạng thái phân quyền cùng nhánh an toàn và nhánh vi phạm.

Bảng 3.2: Đối chiếu oracle so sánh chéo tài khoản và quan hệ MR-A1

Tiêu chí	Oracle so sánh chéo tài khoản	MR-A1 (theo thời gian)
So sánh theo	nhiều tài khoản tại một thời điểm	một tài khoản trước và sau khi hạ quyền
Thông tin xác thực	khác nhau theo tài khoản	giữ cố định và dùng lại
Nội dung kiểm tra	tài khoản không có quyền vẫn truy cập được	quyền đã thu hồi vẫn được chấp nhận

3.3 Bộ ba thao tác và điều kiện tiền đề

Bộ dựng kịch bản kiểm thử chuyển quan hệ MR-A1 thành ca kiểm thử chạy được trên REST API, gọi là kịch bản kiểm thử MR-A1. Đầu vào của bộ dựng là các ca kiểm thử trong kho lưu trữ, gồm cả những ca nhận mã 403 do bước sinh phản hồi 403 có sẵn của EvoMaster tạo ra [2].



Hình 3.2: Không gian trạng thái phân quyền và quan hệ MR-A1

3.3.1 Cấu trúc bộ ba thao tác

Kịch bản kiểm thử MR-A1 gồm các lời gọi khởi tạo tiền đề (nếu có) và bộ ba thao tác chính, thực hiện theo trình tự sau:

1. **Khởi tạo (setup):** các lời gọi đi trước thao tác gốc trong ca kiểm thử ban đầu của nạn nhân nhằm tạo tài nguyên cần thao tác. Bước này không bắt buộc nếu tài nguyên đã có sẵn trong hệ thống.
2. **Thao tác gốc (source):** tài khoản nạn nhân gọi một thao tác ghi nhạy cảm trước khi bị hạ quyền và nhận mã `2xx`.
3. **Thao tác chuyển trạng thái (transition):** tài khoản quản trị hạ vai trò hoặc thu hồi quyền tương ứng của nạn nhân.
4. **Thao tác kiểm chứng (follow-up):** nạn nhân gọi lại đúng thao tác source với cùng tham số, dùng đúng token cũ.

Thao tác follow-up dùng lại token cũ nhờ EvoMaster đăng nhập mỗi tài khoản một lần ở đầu ca kiểm thử, rồi dùng token hoặc cookie phiên đó cho mọi thao tác sau.

3.3.2 Điều kiện tiền đề và giả định trên tài nguyên

Bộ dựng kiểm tra điều kiện tiền đề ngay khi chọn thao tác gốc. Một thao tác ghi được chọn làm source khi có đồng thời hai quan sát sau:

1. Ít nhất một tài khoản khác từng bị từ chối với mã 403 khi gọi thao tác này.
2. Tài khoản nạn nhân từng thực hiện thành công thao tác này với mã `2xx` ở vai trò hiện tại.

Cặp phản hồi 403 và `2xx` này chỉ là dấu hiệu ban đầu, vì lời từ chối có thể đến từ quyền sở hữu tài nguyên hoặc một điều kiện khác. Do follow-up lặp lại source trên cùng tài nguyên, bộ dựng ưu tiên các thao tác cập nhật gọi lại được nhiều lần như `PUT` và `PATCH`. Phương thức `POST` chỉ được xét đến khi không tìm thấy thao tác cập nhật nào phù hợp.

3.3.3 Bộ ba bị hủy và bước đăng nhập lại

Bộ dựng chuyển bộ ba sang bộ phán quyết khi cả ba phản hồi đều hợp lệ. Bộ ba bị hủy và loại khỏi tập đánh giá khi source hoặc transition trả mã ngoài nhóm `2xx`, hoặc khi follow-up nhận một mã khác `2xx` và 403, chẳng hạn `401`. Sau mỗi bộ ba bị hủy, bộ dựng thử một tổ hợp mới gồm tài khoản nạn nhân, tài khoản quản trị và giá trị vai trò thấp hơn.

Trong một số lần thử, mọi tài khoản quản trị đều bị từ chối khi hạ quyền, dù source của nạn nhân vẫn thành công. Tình huống này xảy ra khi pha tìm kiếm đã hạ quyền các tài khoản quản trị nhưng chưa hạ quyền tài khoản nạn nhân. Bộ dựng khi đó dùng quyền quản trị còn hiệu lực của tài khoản nạn nhân để khôi phục vai trò cho tài khoản quản trị rồi thử lại.

Khi follow-up nhận `2xx`, bộ dựng chạy thêm bước đăng nhập lại trước khi chuyển bộ ba sang bộ phán quyết. Bước này kiểm tra thao tác transition có thực sự thu hồi quyền, vì tên endpoint chưa đủ để khẳng định điều đó. Một endpoint có tên gợi ý hạ quyền vẫn có thể nâng quyền, tùy giá trị vai trò gửi kèm.

Ở bước đăng nhập lại, bộ dựng dùng token mới của tài khoản nạn nhân để chạy lại chuỗi lời gọi của nạn nhân trong ca kiểm thử gốc. Chuỗi này gồm các lời gọi khởi tạo dữ liệu và thao tác source. Do EvoMaster dừng chuỗi tại lời gọi tạo tài nguyên đầu tiên thất bại, bộ dựng đọc mã trạng thái của lời gọi cuối cùng được thực thi. Mã 403 tại lời gọi này cho thấy token mới đã mất quyền ghi.

Khi nhận mã 403, bộ dựng chạy lại cùng chuỗi một lần nữa mà không kèm thông tin xác thực để làm phép đối chứng. Nếu lượt gọi nặc danh này cũng bị từ chối với mã 403, phản hồi nhận được bằng token mới chưa chắc phản ánh việc thu hồi quyền. Bộ ba chỉ được giữ lại khi lượt đối chứng trả về một mã khác 403, chẳng hạn mã 401. Mọi trường hợp còn lại, kể cả khi token mới vẫn nhận `2xx`, đều làm bộ ba bị hủy.

Bước đăng nhập lại chạy sau follow-up, vì ở hệ thống giữ một phiên cho mỗi tài khoản, lần đăng nhập mới sẽ vô hiệu hóa token cũ mà follow-up cần. EvoMaster đăng nhập ở đầu mỗi ca kiểm thử, nên bước lấy token mới chạy trong một ca kiểm thử riêng.

Công cụ hộp đen không quan sát trực tiếp được điều kiện tiền đề $Pre(a, T)$, do quy tắc phân quyền này nằm ở tầng logic nội bộ của ứng dụng. Bộ dựng ước lượng điều

kiện này bằng dấu hiệu HTTP ở hai thời điểm: khi chọn source và ở bước đăng nhập lại. Tại bước sau, mã 403 có thể đến từ một lời gọi khởi tạo thay vì từ chính source. Một cảnh báo của Oracle 970 vì vậy là vi phạm MR-A1 khi hai giả định cùng đúng: mã 403 phản ánh quyết định phân quyền, và phiên mới mang quyền hiện hành. Nếu giả định thứ hai sai, tức token mới vẫn mang quyền cũ, bước đăng nhập lại có thể loại nhầm một bộ ba chứa lỗ hổng.

3.3.4 Thuật toán dựng kịch bản kiểm thử MR-A1

Dựa trên cấu trúc bộ ba và các điều kiện hủy nêu trên, bộ dựng duyệt các thao tác chuyển trạng thái đã xếp hạng cùng thao tác ghi ứng viên trong kho lưu trữ. Mỗi tổ hợp được thực thi trong giới hạn số lần thử, kèm bước khôi phục quyền khi thao tác chuyển trạng thái bị từ chối. Các bước của quy trình này được trình bày trong Thuật toán 1.

Trong Thuật toán 1, c_v và c_o là thông tin xác thực của tài khoản nạn nhân v và tài khoản quản trị. CandidateWrites ước lượng $Pre(a, T)$ bằng hai quan sát ở Mục 3.3.2 và loại chính thao tác T. Discarded xét lần lượt source, transition và follow-up theo các điều kiện hủy ở Mục 3.3.3, gồm cả bước đăng nhập lại khi follow-up nhận `2xx`. Bước khôi phục quyền ở Mục 3.3.3 do RestoreOperator thực hiện; lần thử kế tiếp cho biết bước này có thành công hay không.

Thuật toán giới hạn chi phí thực thi bằng ba trần: tổng $N = 20$ lần thử trong cả pha, $K = 5$ lần thử cho mỗi thao tác chuyển trạng thái và $R = 3$ lần khôi phục quyền cho mỗi thao tác chuyển trạng thái. Thuật toán được trình bày ở dạng rút gọn: trong cài đặt, trần K không áp dụng cho thao tác chuyển trạng thái đã dựng được bộ ba và cho ứng viên cuối của danh sách. Với mỗi thao tác ghi ứng viên, bộ ba hợp lệ đầu tiên được đưa vào tập kết quả cho bộ phán quyết, kể cả khi follow-up nhận 403. Bộ dựng sau đó chuyển sang ứng viên kế tiếp, nên một lượt chạy có thể cho nhiều bộ ba.

3.4 Tự động nhận diện và xếp hạng thao tác chuyển trạng thái

Số lần thử của Thuật toán 1 có giới hạn, nên bộ dựng cần thử trước các thao tác có khả năng hạ quyền cao theo một danh sách đã xếp hạng. Một danh sách từ

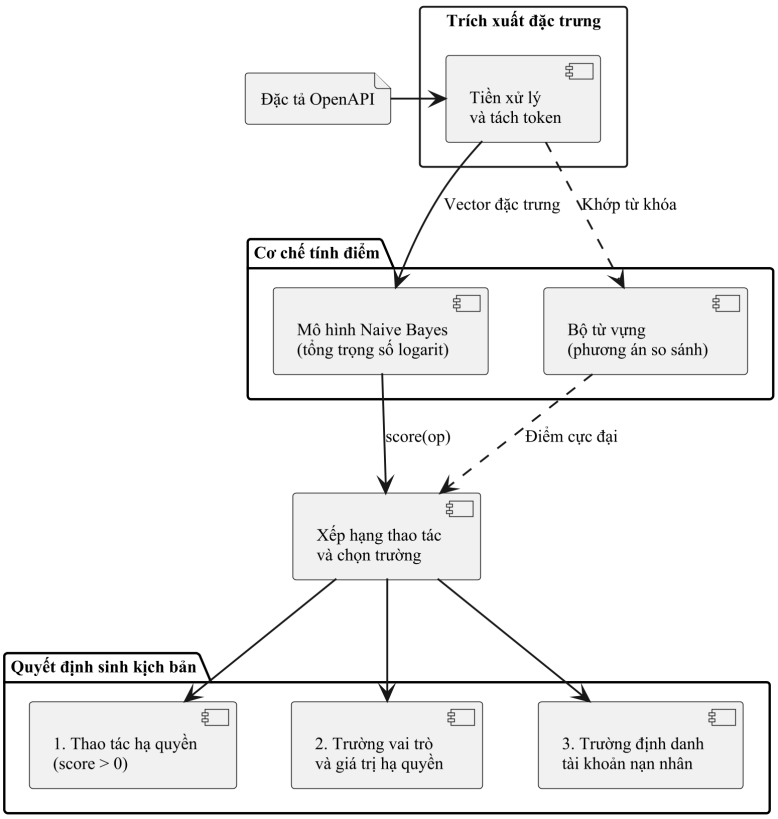
Thuật toán 1 Dựng kịch bản kiểm thử MR-A1

Require: test store $\mathcal{A}$, attempt budgets K and N, restore budget R

Ensure: set $\mathcal{I}$ of performed triples

$n \leftarrow 0$; $\mathcal{I} \leftarrow \emptyset$
for each $T \in$ RankTransitions($\mathcal{A}$) **do**
    $k \leftarrow 0$; $r \leftarrow 0$
    for each $a \in$ CandidateWrites($\mathcal{A}, T$) **do**
        while $k < K$ **and** $n < N$ **do**
            $I \leftarrow [\mathit{setup}(a),\ a(c_v),\ T(c_o, v),\ a(c_v)]$
            Execute(I)
            if $\neg$Discarded(I) **then**
                $\mathcal{I} \leftarrow \mathcal{I} \cup \{I\}$; **break**
            else if T refused with 401 or 403 **and** $r < R$ **then**
                RestoreOperator(c_v); $r \leftarrow r + 1$
            end if
            $k \leftarrow k + 1$; $n \leftarrow n + 1$
        end while
    end for
end for
return $\mathcal{I}$

khóa cố định khó nhận diện thao tác hạ quyền trên một REST API bất kỳ, nên bộ dựng chấm điểm từng thao tác ghi để lập danh sách. Nghiên cứu phát triển hai cơ chế chấm điểm: bộ từ vựng (Mục 3.4.1) là phiên bản ban đầu, sau đó được thay thế bằng mô hình Naive Bayes (Mục 3.4.2). Hình 3.3 tóm tắt quy trình nhận diện.



Hình 3.3: Quy trình nhận diện thao tác hạ quyền từ đặc tả OpenAPI

3.4.1 Tiếp cận dựa trên bộ từ vựng

Bộ từ vựng (lexicon) được học có giám sát từ một tập ngữ liệu thao tác ghi gán nhãn theo ba lớp. Nhãn `R` dành cho thao tác làm giảm quyền của một chủ thể, nhãn `N` cho thao tác không hạ quyền, và nhãn `U` cho trường hợp đặc tả không đủ cơ sở phân loại. Năm quy tắc nhận diện nhãn `R` gồm xóa vai trò hoặc quyền, gỡ chủ thể khỏi vai trò, vô hiệu hóa tài khoản, đặt giá trị vai trò và bỏ vai trò khỏi phiên. Các quy tắc này dựa trên chuẩn kiểm soát truy cập theo vai trò INCITS 359 [14] và chuẩn quản lý danh tính SCIM [16]. Phần huấn luyện của tập ngữ liệu gồm 29 bản đặc tả, với 36 thao tác nhãn `R` và 441 thao tác nhãn `N`. Nhãn do mô hình ngôn ngữ lớn gán tự động qua hai lượt, sau đó một phần nhãn được soát lại

thủ công.

Hai nhóm token đường dẫn và token tham số được trích xuất lần lượt từ URL và tên tham số của các thao tác mang nhãn `R`. Quá trình tiền xử lý tách chuỗi theo quy tắc camelCase cùng các ký tự phân tách đặc biệt, sau đó chuyển toàn bộ về chữ thường. Các token thuần chữ số hoặc có độ dài dưới bốn ký tự bị loại bỏ. Điểm của mỗi từ w được tính theo Công thức (3.2):

$$\mathrm{s}(w) = \frac{d_R + 1}{d_R + d_N + 2} \times \frac{n_{cc}}{n_{cc} + 1}. \tag{3.2}$$

Trong đó, d_R và d_N là số thao tác nhãn `R` và `N` chứa w, còn n_{cc} là số nhà cung cấp API có thao tác nhãn `R` chứa w. Thừa số $n_{cc}/(n_{cc}+1)$ giảm điểm của token xuất hiện ở một nhà cung cấp duy nhất. Điểm của một endpoint bằng điểm lớn nhất của các token khớp với chuỗi định danh của nó. Bộ từ vựng thu được gồm 53 token đường dẫn và 45 token tham số.

Chuỗi định danh của mỗi endpoint được tổng hợp từ đường dẫn, phương thức HTTP và danh sách các trường trong thân yêu cầu (*request body*). Thành phần này giúp nhận diện những thao tác hạ quyền mà vai trò được truyền qua dữ liệu gửi kèm thay vì thể hiện trực tiếp trên URL. Chẳng hạn, thao tác `PATCH /users/{id}` có thể vô hiệu hóa một tài khoản qua trường `enabled` trong thân yêu cầu, trong khi URL không chứa từ nào về quyền. Khi hai endpoint cùng điểm, bộ từ vựng ưu tiên endpoint có tham số định danh tài khoản người dùng, rồi theo thứ tự `PUT`, `PATCH`, `POST`, `DELETE`.

Hạn chế chính của bộ từ vựng là cách lấy điểm cực đại: điểm của endpoint chỉ dựa vào một token, các token khác bị bỏ qua. Trên đặc tả của netbox, token `outside` (điểm 0,375, lấy từ trường `nat_outside`) thắng token `username` (0,171) và đẩy thao tác hạ quyền xuống hạng 77. Mô hình Naive Bayes cộng bằng chứng từ mọi đặc trưng có mặt. Mô hình này còn dùng chung cho xếp hạng thao tác, chọn trường vai trò và giá trị hạ quyền, nên được chọn thay cho bộ từ vựng. Chương 5 so sánh hai cách xếp hạng.

3.4.2 Tiếp cận dựa trên mô hình Naive Bayes

Naive Bayes là bộ phân loại xác suất dựa trên định lý Bayes, giả định các đặc trưng độc lập có điều kiện khi biết nhãn. Mô hình học từ một tập ngữ liệu lớn hơn, gồm 10.381 thao tác ghi của 230 REST API. Nhãn nhị phân của mỗi thao

tác cho biết thao tác đó có cấp, đổi hoặc gỡ vai trò hay quyền của một tài khoản đã tồn tại hay không. Hai lượt gán độc lập của mô hình ngôn ngữ lớn cho nhãn khởi đầu. Người gán xem 602 thao tác: mọi thao tác mà hai lượt bất đồng, 165 trong 173 thao tác hai lượt cùng gọi chuyển trạng thái, và 371 thao tác bốc ngẫu nhiên trong 10.142 thao tác hai lượt cùng gọi thông thường. Thao tác nào người gán khác cả hai lượt thì được xem lại và chốt riêng. 9.771 thao tác hai lượt cùng gọi thông thường mà chưa ai xem giữ nhãn thông thường. Tập nhãn cuối có 209 thao tác thuộc lớp chuyển trạng thái. Mô hình đi kèm công cụ học trên toàn bộ tập này và được chấm trên một tập đặc tả riêng (Mục 5.3). Ngoài xếp hạng thao tác chuyển trạng thái, bộ dựng dùng mô hình để chọn trường vai trò, trường định danh tài khoản nạn nhân và giá trị hạ hoặc nâng quyền.

Với mỗi đặc trưng t, mô hình tính một trọng số logarit theo tần suất của t ở hai lớp. Điểm của một thao tác op được tính theo Công thức (3.3):

$$w(t) = \ln \frac{d_T + 1}{N_T + 2} - \ln \frac{d_N + 1}{N_N + 2}, \qquad \text{score}(op) = \ln \frac{N_T}{N_N} + \sum_{t \in op} w(t). \tag{3.3}$$

Trong đó, d_T và d_N là số thao tác thuộc lớp chuyển trạng thái và lớp thông thường có chứa t, còn N_T và N_N là tổng số thao tác của hai lớp. Thao tác có $\text{score}(op) > 0$ được coi là ứng viên chuyển trạng thái. Trường hợp hai thao tác có điểm số bằng nhau, bộ dựng ưu tiên theo thứ tự xuất hiện ban đầu của chúng trong đặc tả OpenAPI. Điểm số chỉ dùng để xếp hạng các ứng viên; nó chưa được hiệu chuẩn để đọc như một xác suất hậu nghiệm.

Không gian đặc trưng gồm ba nhóm: token đường dẫn, tên trường hoặc tham số, và phương thức HTTP. Mô hình giữ những đặc trưng xuất hiện trong thao tác chuyển trạng thái của ít nhất hai nhà cung cấp API, và đặc trưng được giữ vẫn có thể mang trọng số âm. Điểm số được tính bằng tổng trọng số của các đặc trưng xuất hiện trong thao tác. Khác với biến thể Bernoulli Naive Bayes đầy đủ vốn xét cả xác suất của đặc trưng không xuất hiện, mô hình ở đây bỏ qua các đặc trưng này. Bộ dựng và bộ phán quyết dùng chung quy tắc $\text{score}(op) > 0$, nên bộ phán quyết luôn coi thao tác ở giữa mỗi bộ ba là thao tác chuyển trạng thái.

Trong kiểm chứng chéo ở Mục 5.3, Naive Bayes để tối đa 3 ứng viên đứng trước thao tác đích, còn bộ từ vựng có một lần để 76 ứng viên đứng trước. Giả định độc lập của mô hình bị vi phạm một phần ở các token thường đi cùng nhau, như các token về người dùng, tên đăng nhập và thư điện tử.

Chương 4

Cài đặt thực nghiệm và tích hợp hệ thống

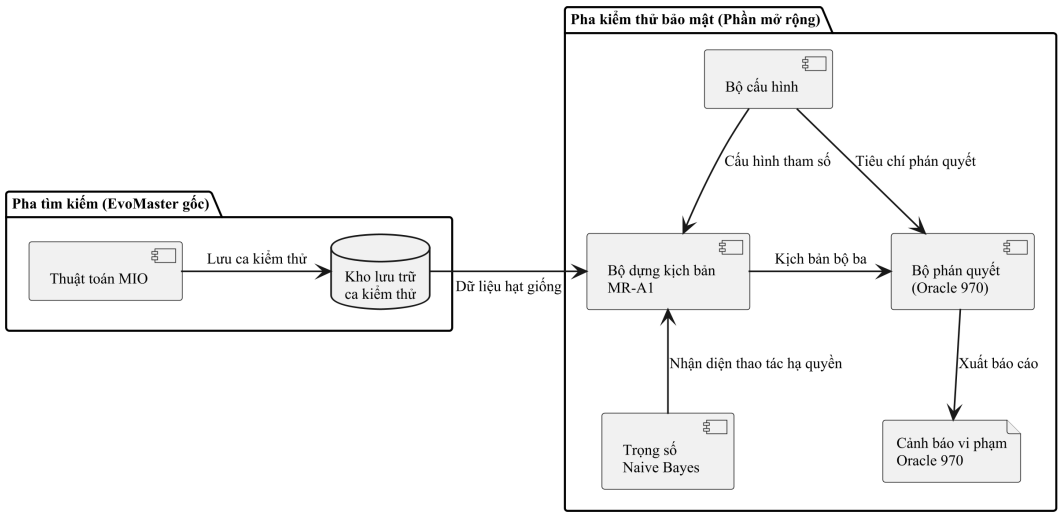
Chương này trình bày phương án cài đặt Oracle 970 trong EvoMaster và thiết lập môi trường thực nghiệm nhằm đánh giá quan hệ MR-A1 trên các ứng dụng có cơ chế thay đổi quyền.

4.1 Tích hợp Oracle 970 vào mã nguồn EvoMaster

Phần mở rộng của Oracle 970 được phát triển bằng Kotlin trực tiếp trên mã nguồn EvoMaster phiên bản 6.1.1 [9], cùng ngôn ngữ với phần lõi của công cụ. EvoMaster được cấu hình chạy ở chế độ hộp đen, đọc tài liệu đặc tả OpenAPI và gửi các yêu cầu HTTP tới ứng dụng mục tiêu đang chạy. Các thành phần mở rộng được bổ sung vào quy trình kiểm thử mà vẫn giữ nguyên thuật toán tìm kiếm MIO (Many Independent Objective) của EvoMaster.

Hình 4.1 mô tả ba thành phần của phần mở rộng và vị trí của chúng trong chu trình EvoMaster. Bộ cấu hình khai báo mã lỗi của Oracle 970 cùng các tùy chọn vận hành. Từ kho lưu trữ ca kiểm thử, bộ dựng kịch bản kiểm thử xếp hạng thao tác chuyển trạng thái rồi dựng kịch bản kiểm thử MR-A1. Bộ phán quyết nhận kịch bản này, đọc mã trạng thái HTTP của bộ ba thao tác và kết luận có vi phạm hay không. Công cụ đóng gói sẵn trọng số của mô hình Naive Bayes, nên các lượt chạy không phải huấn luyện lại.

Các thành phần trên chạy trong pha kiểm thử bảo mật của EvoMaster [3], sau pha tìm kiếm. Các bộ sinh ca kiểm thử bảo mật trong pha này dùng chung một quỹ thời gian, được tính theo tỷ lệ so với thời gian tìm kiếm. Khi hết thời gian phân bổ, các bộ sinh chạy sau cùng có thể bị bỏ qua, nên bộ dựng kịch bản MR-A1 được đặt ngay sau bước sinh phản hồi 403 có sẵn của EvoMaster. Vị trí này còn cho phép bộ dựng chọn thao tác gốc trong số các thao tác vừa bị từ chối ở bước đó.



Hình 4.1: Ba thành phần phần mở rộng trong chu trình EvoMaster

4.2 Xây dựng tập benchmark và ứng dụng mục tiêu

Tập benchmark gồm bốn ứng dụng mục tiêu (Bảng 4.1). Hai ứng dụng mang lỗi gieo: một ứng dụng xây dựng riêng cho thực nghiệm và một bản fork. Hai ứng dụng mã nguồn mở mang lỗi thật: pyLoad với lỗ hổng đã công bố, và artifact-keeper với một lỗi mà nhà phát triển đã vá. Mỗi ứng dụng cần có thao tác hạ quyền qua REST API, thao tác ghi dành cho quản trị viên và lỗ hổng phân quyền có trạng thái. Các phiên bản VAmPI và crAPI đã khảo sát đều thiếu endpoint hạ quyền như vậy, nên hai ứng dụng đầu phải được xây dựng hoặc gieo lỗi riêng.

Mỗi ứng dụng có hai biến thể: biến thể có lỗ hổng dùng để đo khả năng phát hiện, biến thể đã vá dùng để đo báo động giả. Bản vá của reports-api, vampi-seeded và pyLoad do tác giả viết; bản vá của artifact-keeper là bản vá của nhà phát triển.

4.2.1 Ứng dụng reports-api

Ứng dụng reports-api là dịch vụ REST tối giản viết bằng Spring Boot 3 và Java 17, dùng để kiểm tra Oracle 970 trên các trường hợp đã biết trước kết quả. Máy chủ cấp token JWT khi đăng nhập [25] và yêu cầu vai trò quản trị cho các thao tác ghi trên báo cáo cũng như thao tác hạ quyền `PUT /users/{username}/role`. Ở biến thể có lỗ hổng, máy chủ đọc vai trò từ claim (trường thông tin mang trong JWT) thay vì tra vai trò hiện tại trong cơ sở dữ liệu. Hệ quả là tài khoản đã bị hạ quyền vẫn ghi được dữ liệu bằng token cấp trước khi đổi vai trò. Hai biến thể chạy

Bảng 4.1: Các ứng dụng mục tiêu

Mục tiêu	Nền tảng	Số thao tác	Nguồn lỗi	Xác thực
reports-api	Spring Boot 3 / Java 17	9	Lỗi gieo có kiểm soát	JWT Bearer
vampi-seeded	Python / Flask	16	Lỗi gieo trên bản fork	JWT Bearer
pyLoad	Python / Flask	107	CVE-2026-41133	Cookie phiên
artifact-keeper	Rust	389	Lỗi #1821, nhà phát triển đã vá	JWT Bearer

từ cùng một bản đóng gói và khác nhau ở giá trị cấu hình chọn nguồn đọc vai trò.

4.2.2 Ứng dụng vampi-seeded

Ứng dụng vampi-seeded kiểm tra Oracle 970 trên mã nguồn và quy ước đặt đường dẫn của một dự án bên thứ ba. Ứng dụng này là bản fork (bản sao tách nhánh) của dự án mã nguồn mở VAmPI, được bổ sung cơ chế phân quyền có lỗ hổng với thay đổi nhỏ nhất trên mã gốc. Các endpoint bổ sung theo quy ước đặt đường dẫn của VAmPI, gồm thao tác hạ quyền `PUT /users/v1/{username}/role` và thao tác ghi cập nhật dữ liệu sách. Do dự án gốc chưa có chức năng đổi vai trò, endpoint hạ quyền được định nghĩa mới. Cơ chế gieo lỗi ghi cờ quản trị vào claim của token khi đăng nhập, và một biến môi trường chọn giữa hai biến thể. Thời hạn token được nâng từ 60 lên 3600 giây để token không hết hạn giữa một bộ ba.

4.2.3 Ứng dụng pyLoad

Khác với hai ứng dụng trên, pyLoad là trình quản lý tải viết bằng Python/Flask, mang lỗ hổng CVE-2026-41133 (GHSA-66hx-chf7-3332) do bên thứ ba công bố độc lập [26]. Lỗ hổng này thuộc loại CWE-613, điểm CVSS 8,8, và bắt nguồn từ việc pyLoad sao chép vai trò vào phiên lúc đăng nhập [4]. Khi vai trò thay đổi, ứng

dụng ghi vai trò mới vào cơ sở dữ liệu nhưng giữ nguyên phiên đã cấp.

Thực nghiệm dùng pyLoad bản dev97, kèm một biến thể đã vá do tác giả xây dựng để đối chứng. Thông báo lỗ hổng ghi bản vá ở phiên bản dev98, song hàm kiểm tra xác thực vẫn giữ nguyên từ dev97 tới dev101 và trên nhánh phát triển tại thời điểm thực nghiệm. Biến thể đã vá tra lại vai trò hiện tại từ cơ sở dữ liệu ở cả hai nhánh kiểm tra phân quyền, thay vì hủy phiên.

So với cấu hình mặc định, môi trường thử nghiệm trên pyLoad có các điều chỉnh thuộc ba phạm vi: tầng ứng dụng, dữ liệu đặc tả và cấu hình công cụ. Ở tầng ứng dụng, kiểm tra chống giả mạo yêu cầu liên trang (CSRF) được tắt trên cả hai biến thể, biến thể đã vá do tác giả viết, và một endpoint khôi phục dữ liệu tài khoản được bổ sung. Ứng dụng còn được tạo sẵn thêm tài khoản quản trị, vì MR-A1 cần một tài khoản nạn nhân và một tài khoản quản trị, trong khi bản cài đặt mặc định chỉ có một tài khoản. Về dữ liệu đầu vào, nhóm endpoint `/json/*` được bổ sung đặc tả OpenAPI soạn thảo thủ công. Khi khởi chạy công cụ, hai endpoint làm dừng tiến trình pyLoad bị loại khỏi phạm vi kiểm thử. Tệp chứa toàn bộ logic phân quyền của biến thể có lỗ hổng giữ nguyên mã upstream.

4.2.4 Ứng dụng artifact-keeper

artifact-keeper là kho lưu trữ artifact phần mềm mã nguồn mở viết bằng Rust [27]. Ở commit có lỗi (`e56d3aa7e2`), máy chủ ghi cờ `is_admin` vào claim của JWT lúc đăng nhập và đọc lại cờ này từ claim mà không tra cơ sở dữ liệu. Quản trị viên bị hạ quyền vì vậy vẫn giữ quyền quản trị tới khi token hết hạn sau 30 phút. Nhà phát triển sửa lỗi này ở commit `e64cc5c608` (issue #1821): token cấp trước lần thay đổi quyền bị từ chối với mã 401. Thao tác hạ quyền là `PATCH /api/v1/users/{id}` với trường `is_admin` trong thân yêu cầu. Cả hai commit được kiểm tra thủ công qua HTTP trước khi chạy công cụ.

Phần tác giả thay đổi chỉ gồm cấu hình dựng container và đặc tả OpenAPI. Endpoint `DELETE /users/{id}` được bỏ khỏi đặc tả để công cụ không xóa các tài khoản kiểm thử; mã ứng dụng, lỗi, bản vá và phần còn lại của đặc tả là của nhà phát triển. Mã nguồn của bộ dựng, bộ phán quyết và mô hình Naive Bayes không chứa chuỗi nào riêng của ứng dụng này, và đặc tả của nó không thuộc tập ngữ liệu huấn luyện.

Một số sửa đổi chung của bộ dựng được viết trong lúc chạy công cụ trên pyLoad và artifact-keeper. Kết quả trên hai ứng dụng này vì vậy cho thấy công cụ phát hiện được hai lỗi thật, nhưng không dùng để ước lượng tỷ lệ phát hiện trên ứng dụng chưa từng chạy.

4.3 Thiết lập thực nghiệm

Mọi lượt chạy đánh giá đều bật các oracle thử nghiệm của EvoMaster, và dữ liệu của ứng dụng được khởi tạo lại trước mỗi lượt để các lượt độc lập với nhau. Bảng 4.2 ghi thời gian tìm kiếm, quỹ thời gian của pha kiểm thử bảo mật (tính theo tỷ lệ của thời gian tìm kiếm) và số seed của từng ứng dụng. EvoMaster chạy trước một ca kiểm thử cho mỗi thao tác với mỗi tài khoản, nên artifact-keeper, với 389 thao tác, được chạy với thời gian tìm kiếm dài hơn và tốc độ gửi yêu cầu cao hơn. Kết quả thực nghiệm và phân tích định lượng của các lượt chạy này được trình bày trong Chương 5.

Bảng 4.2: Thiết lập chạy theo ứng dụng

Ứng dụng	**Thời gian tìm kiếm**	**Yêu cầu mỗi phút**	**Quỹ pha bảo mật**	**Seed mỗi biến thể**
reports-api	150 giây	120	0,6	20
vampi-seeded	150 giây	120	0,6	20
pyLoad	10 phút	200	0,6	5
artifact-keeper	20 phút	600	0,25	5 bản lỗi, 3 bản vá

Chương 5

Thực nghiệm và đánh giá kết quả

Chương này đánh giá khả năng phát hiện vi phạm, mức báo động giả và khả năng nhận diện thao tác chuyển trạng thái của Oracle 970. Kết quả được so sánh với baseline, tức EvoMaster v6.1.1 chạy với các oracle phân quyền sẵn có, không kèm Oracle 970.

5.1 Thiết kế thực nghiệm

Để đánh giá khả năng phát hiện vi phạm và mức báo động giả của Oracle 970, thực nghiệm dùng thiết kế cặp đối chứng trên hai biến thể của mỗi ứng dụng mục tiêu. Biến thể có lỗ hổng dùng để đo tỷ lệ phát hiện vi phạm phân quyền có trạng thái. Trên biến thể đã vá, mọi cảnh báo phát sinh đều được tính là báo động giả (false positive, FP). Tổng cộng 98 lượt chạy ở chế độ hộp đen được thực hiện theo thiết lập ở Mục 4.3, gồm 50 lượt trên biến thể có lỗ hổng và 48 lượt trên biến thể đã vá. Mọi lượt dùng cùng một bản đóng gói của công cụ.

Mỗi lượt chạy ghi lại mã trạng thái HTTP của các bộ ba đã thực thi, nên số lượt dựng được bộ ba và số lượt báo vi phạm được thống kê riêng. Ngoài 98 lượt chạy trên các ứng dụng mục tiêu, khả năng xếp hạng thao tác chuyển trạng thái được đánh giá ngoại tuyến trên các tập đặc tả OpenAPI riêng (Mục 5.3).

5.2 Hiệu quả phát hiện và độ chính xác

Bảng 5.1 tổng hợp số lượt dựng được bộ ba và số lượt báo vi phạm trên từng biến thể. Cột cuối của bảng ghi khoảng tin cậy Clopper–Pearson 95% của tỷ lệ phát hiện trên biến thể có lỗ hổng [28]. Khoảng này tính chính xác từ phân phối nhị thức, nên độ phủ đạt ít nhất 95% kể cả với mẫu 5 lượt.

Tính chung 4 ứng dụng, Oracle 970 phát hiện vi phạm ở 36/50 lượt trên biến thể có lỗ hổng và không báo vi phạm ở lượt nào trong 48 lượt trên biến thể đã vá. Kiểm định chính xác Fisher một phía cho thấy tỷ lệ báo vi phạm khác nhau giữa

Bảng 5.1: Kết quả phát hiện và báo động giả theo lượt chạy

Ứng dụng	**Biến thể**	**Lượt có bộ ba**	**Lượt báo vi phạm**	**Clopper–Pearson 95% (%)**
reports-api	có lỗ hổng	19/20	19/20	[75,1; 99,9]
reports-api	đã vá	19/20	0/20	—
vampi-seeded	có lỗ hổng	7/20	7/20	[15,4; 59,2]
vampi-seeded	đã vá	6/20	0/20	—
pyLoad	có lỗ hổng	5/5	5/5	[47,8; 100]
pyLoad	đã vá	5/5	0/5	—
artifact-keeper	có lỗ hổng	5/5	5/5	[47,8; 100]
artifact-keeper	đã vá	0/3	0/3	—

2 biến thể ở cả 4 ứng dụng ($p < 0{,}001$ với reports-api, $p \approx 0{,}0042$ với vampi-seeded, $p \approx 0{,}0040$ với pyLoad, $p \approx 0{,}018$ với artifact-keeper) [29].

Bảng 5.2 phân loại lượt đã thực hiện ít nhất một bộ ba hợp lệ theo nhãn TP (phát hiện đúng), FN (bỏ sót), FP (báo động giả) và TN (âm tính thật). Lượt không dựng được bộ ba được đếm ở cột riêng; trên biến thể có lỗ hổng, đó là lần bỏ sót của quy trình đầu-cuối.

Bảng 5.2: Phân loại kết quả theo lượt chạy

Ứng dụng	**TP**	**FN, có bộ ba**	**Có lỗ hổng, không bộ ba**	**FP**	**TN**	**Đã vá, không bộ ba**
reports-api	19	0	1	0	19	1
vampi-seeded	7	0	13	0	6	14
pyLoad	5	0	0	0	5	0
artifact-keeper	5	0	0	0	0	3
Tổng	36	0	14	0	30	18

Xét riêng các lượt dựng được bộ ba, Oracle 970 báo vi phạm ở cả 36 lượt trên biến

thể có lỗ hổng và không phát sinh cảnh báo nào trong 30 lượt trên biến thể đã vá. Khoảng tin cậy Clopper–Pearson 95% của tỷ lệ báo động giả này là [0; 11,6]%. Mỗi lượt dựng được bộ ba có đúng một bộ ba, với chuỗi mã trạng thái `200/200/200` trên biến thể có lỗ hổng và `200/200/403` trên biến thể đã vá. Tỷ lệ phát hiện tính trên các lượt có bộ ba gần như là hệ quả của định nghĩa, vì tiêu chí hợp lệ của bộ ba dùng chính mã follow-up mà bộ phán quyết đọc. Phép đo độc lập là số báo động giả trên biến thể đã vá.

Trên bản vá của artifact-keeper, cả 3 lượt đều không dựng được bộ ba. Ở 2 lượt, thao tác hạ quyền thành công nhưng token cũ nhận mã 401 ở bước follow-up; mã 401 nằm ngoài phán quyết (Bảng 3.1), nên bộ ba bị hủy. Ở lượt còn lại, thao tác hạ quyền không lần nào trả mã `2xx`. Ba lượt này không có báo động giả, nhưng cũng không được tính là âm tính thật.

Toàn bộ 14 lượt bỏ sót đều không dựng được bộ ba hợp lệ để kích hoạt bộ phán quyết. Nguyên nhân của các lượt này được phân tích tại Mục 5.5.

5.3 Hiệu quả xếp hạng thao tác chuyển trạng thái

Mô hình Naive Bayes đi kèm công cụ được chấm trên 237 bản đặc tả không tham gia huấn luyện, gồm 8.674 thao tác ghi. Trong 153 thao tác chuyển trạng thái của tập này, 117 thao tác (76,5%; khoảng Clopper–Pearson 95% là [68,9; 82,9]) có điểm dương [28]. Ngược lại, 763 trong 880 thao tác có điểm dương mang nhãn thông thường (86,7%), nên điểm dương chỉ đủ để chọn ứng viên, chưa đủ để kết luận một thao tác hạ quyền. Người gán xem 368 thao tác bốc ngẫu nhiên trong số các thao tác mà hai lượt gán cùng gọi thông thường và không thấy thao tác chuyển trạng thái nào. Từ đó, số thao tác chuyển trạng thái bị sót trong 8.014 thao tác chưa ai xem không quá 80 (Clopper–Pearson 95%); nếu cả 80 thao tác đó đều có điểm không dương, độ bắt được giảm xuống 50,2%.

Phép so sánh với bộ từ vựng dùng kiểm chứng chéo 5 phần (5-fold) phân theo nhà cung cấp, lặp lại 4 lần, trên 10.381 thao tác ghi của tập huấn luyện với nhãn của một vòng gán trước (199 thao tác chuyển trạng thái). Ở mỗi lần lặp, cả hai phương pháp được học lại trên phần huấn luyện. Kết quả được tổng hợp trên 47 bản đặc tả có thao tác chuyển trạng thái, gọi là thao tác đích. Một bộ lọc ứng viên dùng chung, giữ các thao tác xác định được trường vai trò và giá trị hạ quyền, giữ lại

thao tác đích ở 21 đến 25 trong 47 bản đặc tả. Bảng 5.3 ghi kết quả của 4 lần lặp; các cột thứ hạng chỉ tính trên những bản đặc tả có thao tác đích qua bộ lọc.

Bảng 5.3: Kết quả xếp hạng thao tác đích trên 47 bản đặc tả

Phương pháp	**Qua bộ lọc**	**Hạng 1**	**Trong 4 hạng đầu**	**Số ứng viên đứng trước (tối đa)**
Bộ từ vựng	22 / 21 / 22 / 25	18 / 19 / 19 / 20	22 / 21 / 22 / 23	2 / 2 / 2 / 76
Naive Bayes	22 / 21 / 22 / 25	16 / 16 / 17 / 18	22 / 21 / 22 / 25	1 / 2 / 1 / 3

Ở hạng 1, bộ từ vựng hơn Naive Bayes 2 đến 3 bản đặc tả ở cả 4 lần lặp. Ở 3 lần lặp đầu, cả hai phương pháp đưa mọi thao tác đích đã qua bộ lọc vào 4 hạng đầu; ở lần lặp thứ tư, Naive Bayes giữ được cả 25 bản, bộ từ vựng 23 bản. Khác biệt lớn nhất nằm ở trường hợp xấu nhất: bộ từ vựng có một lần để 76 ứng viên đứng trước thao tác đích (đặc tả netbox, Mục 3.4.2), còn Naive Bayes để nhiều nhất 3. Không chênh lệch nào giữa hai phương pháp đạt mức ý nghĩa $p < 0{,}05$. Naive Bayes được chọn vì thao tác đích luôn nằm trong tầm thử của bộ dựng. Với trần 20 lần thử cho cả pha và 5 lần cho mỗi thao tác chuyển trạng thái chưa dựng được bộ ba (Mục 3.3.4), bộ dựng thử tới ít nhất 4 ứng viên đầu nếu pha chưa hết thời gian.

Trên đặc tả của các ứng dụng mục tiêu, mô hình đặt thao tác chuyển trạng thái ở hạng 1 trên reports-api, vampi-seeded và pyLoad. Trên artifact-keeper, thao tác này đứng hạng 4 trong 5 ứng viên qua bộ lọc; bộ dựng thử lần lượt các ứng viên và dựng được bộ ba trên ứng viên thứ tư ở cả 5 lượt chạy trên biến thể có lỗ hổng.

Bộ từ vựng cũng được so với danh sách từ khóa cố định gồm 8 từ đường dẫn và 9 từ tham số của phiên bản đầu của bộ dựng. Trên 6 bản đặc tả có thao tác hạ quyền thuộc phần kiểm tra của tập ngữ liệu bộ từ vựng, danh sách cố định không chọn đúng thao tác này ở bản nào (0/6). Bộ từ vựng xếp thao tác này ở hạng 1 trên 4/6 bản và trong 8 hạng đầu trên cả 6 bản. Sáu bản đặc tả này đã được dùng trong lúc phát triển bộ từ vựng.

5.4 Nghiên cứu tình huống trên lỗi thật

pyLoad và artifact-keeper là hai ứng dụng mà lỗi, mã ứng dụng và phần lớn đặc tả không do tác giả viết.

Trên pyLoad, công cụ tự nhận diện thao tác chuyển trạng thái `POST /api/set_user_permission` từ đặc tả, xếp hạng 1 trong 3 ứng viên qua bộ lọc. Ở cả 10 lượt chạy, bộ ba dùng source `POST /json/status`, nạn nhân là tài khoản `pyload` và tài khoản quản trị là `alice`; mỗi lượt cần một lần khôi phục quyền cho tài khoản quản trị trước khi dựng được bộ ba. Oracle 970 báo vi phạm ở 5/5 lượt trên biến thể có lỗ hổng và 0/5 lượt trên biến thể đã vá. Thao tác hạ quyền gửi `permission=1` cùng một giá trị `role` do pha tìm kiếm để lại, luôn khác 0, giá trị mà pyLoad dành cho quản trị viên. Công cụ không tự suy ra giá trị `role` này; bước đăng nhập lại xác nhận quyền đã mất ở từng lượt. Vì pyLoad xác thực bằng cookie phiên thay cho Bearer token, lỗ hổng được xác nhận thêm bằng kiểm tra HTTP thủ công trên cả 2 biến thể.

Với artifact-keeper, công cụ tự chọn thao tác `PATCH /api/v1/users/{id}`, đặt trường `is_admin` thành `false`, và dùng source `POST /api/v1/admin/analytics/snapshot`, một thao tác chỉ dành cho quản trị viên. Ở commit có lỗi, chuỗi mã của bộ ba là `200/200/200` và Oracle 970 báo vi phạm ở 5/5 lượt. Bản vá của nhà phát triển trả mã 401 cho token cũ, nên không lượt nào dựng được bộ ba (Mục 5.2). Với cùng công cụ và cùng cấu hình chạy, mã nguồn của nhà phát triển trước và sau khi sửa lỗi cho hai kết quả khác nhau ($p \approx 0{,}018$), và phép đối chứng này không dựa vào một bản vá do tác giả viết.

5.5 Phân tích nguyên nhân bỏ sót và giới hạn

Toàn bộ 14 lượt bỏ sót trên biến thể có lỗ hổng được chia thành 2 nhóm. Nhóm thứ nhất gồm 8 lượt (1 trên reports-api và 7 trên vampi-seeded), trong đó bộ dựng đã thử dựng bộ ba nhưng mọi lần thử đều bị hủy theo các điều kiện ở Mục 3.3.3. Ở nhóm thứ hai, gồm 6 lượt còn lại trên vampi-seeded, bộ dựng không thực hiện lần thử nào; nhật ký ở mức mặc định của EvoMaster không ghi lý do cụ thể.

Pha tìm kiếm có thể làm thay đổi trạng thái ứng dụng trước khi bộ dựng chạy, qua đó làm giảm số bộ ba dựng được. Chẳng hạn, tài khoản nạn nhân có thể bị hạ

quyền ngay trong pha này, làm source nhận mã 403, hoặc bị xóa, làm bước khôi phục quyền mất tác dụng. Trên vampi-seeded, các ca kiểm thử do pha tìm kiếm sinh ra gọi thao tác xóa tài khoản `DELETE /users/v1/{username}` nhiều lần trên chính các tài khoản kiểm thử. Các tác dụng phụ này làm giảm số lượt phát hiện; trên các bộ ba hợp lệ đã quan sát, chúng chưa gây ra phán quyết sai nào.

Về mặt thống kê, quy mô mẫu còn nhỏ: 20 seed cho mỗi biến thể của reports-api và vampi-seeded, 5 seed cho mỗi biến thể của pyLoad, 5 seed cho biến thể có lỗi và 3 seed cho bản vá của artifact-keeper. Kiểm định Fisher giả định 2 mẫu độc lập, trong khi 2 biến thể dùng chung seed, nên các giá trị p chủ yếu mang ý nghĩa tham khảo.

Phán quyết dựa trên mã trạng thái HTTP có giới hạn riêng: mã `2xx` chưa chứng minh trạng thái dữ liệu nghiệp vụ sau thao tác, còn mã 403 không nêu nguyên nhân từ chối. Theo phân tích mã nguồn reports-api, mã 403 ở bước đăng nhập lại phần lớn đến từ lời gọi tạo báo cáo đứng trước source, vốn cũng dành cho quản trị viên. Mã này vẫn cho thấy token mới đã mất quyền ghi, dù lời gọi bị từ chối khác thao tác đang xét. Ngoài ra, mã từ chối được cố định là 403 và chưa cấu hình được. Bản vá của artifact-keeper từ chối token cũ bằng mã 401, nên công cụ không dựng được bộ ba nào trên bản vá này.

Kết quả kiểm chứng chéo tại Mục 5.3 chỉ là đánh giá sơ bộ, do 47 bản đặc tả này đã được sử dụng qua nhiều vòng lựa chọn mô hình. Trong 10.381 thao tác ghi của tập huấn luyện, người gán xem 602 thao tác; 9.771 thao tác mà hai lượt gán của mô hình ngôn ngữ lớn cùng gọi thông thường chưa ai xem. Trên tập ngữ liệu của bộ từ vựng, ở phần hai lượt gán của mô hình cho cùng nhãn, hệ số kappa có trọng số so với nhãn do người gán độc lập chỉ đạt 0,369. Phép chấm trên 237 bản đặc tả không tham gia huấn luyện chỉ áp dụng cho mô hình Naive Bayes, chưa so sánh với bộ từ vựng. Nhãn của tập này do hai lượt gán của mô hình ngôn ngữ lớn tạo ra và chỉ được người gán xem lại một phần.

Khả năng khái quát hóa bị giới hạn ở một lớp lỗ hổng phân quyền có trạng thái và một quan hệ MR-A1. Ba ứng dụng reports-api, vampi-seeded và artifact-keeper cùng đọc quyền cũ từ claim của JWT; ở artifact-keeper, đó là mã của nhà phát triển. pyLoad lưu vai trò trong phiên. Đường dẫn thao tác hạ quyền trên vampi-seeded được đặt khi gieo lỗi, vì vậy kết quả nhận diện trên ứng dụng này chưa đại diện cho đặc tả bên thứ ba. Tỷ lệ phát hiện 5/5 trên pyLoad đạt được trong môi

trường thử nghiệm có các điều chỉnh tại Mục 4.2.3 và với biến thể đã vá do tác giả viết, nên chưa áp dụng trực tiếp cho bản cài đặt mặc định của pyLoad. pyLoad và artifact-keeper cũng là ứng dụng dùng trong lúc phát triển bộ dựng (Mục 4.2.4), nên khóa luận không ước lượng tỷ lệ phát hiện trên ứng dụng chưa từng chạy.

5.6 Đối chứng với baseline và kiểm tra upstream

Phép đối chứng đánh giá khả năng dựng và phán quyết chuỗi MR-A1 của baseline và của các phiên bản mới hơn của EvoMaster. Các oracle phân quyền của baseline thuộc nhóm oracle so sánh chéo tài khoản của Şahin và cộng sự, chỉ đối chiếu quyền giữa các tài khoản tại một thời điểm [3]. Chuỗi hạ quyền rồi dùng lại token cũ theo MR-A1 vì thế nằm ngoài các kịch bản mà baseline phán quyết.

Toàn bộ 20 lượt chạy baseline trên biến thể có lỗ hổng của vampi-seeded đều không sinh được bộ ba thao tác `[source/transition/follow-up]` dùng lại token cũ. Dù vậy, ở 17/20 lượt, baseline vẫn ghi nhận lỗi F209, là lỗi lộ dấu vết ngăn xếp (stack trace) trong phản hồi HTTP. Pha kiểm thử bảo mật của baseline vì vậy vẫn chạy trong các lượt đó, nên việc thiếu bộ ba không do pha này bị bỏ qua.

Bản phát hành EvoMaster v6.2.0 được chạy với 1 seed trong thời lượng 180 giây trên biến thể có lỗ hổng của reports-api và vampi-seeded. Các lớp lỗi phát sinh được ghi trong Bảng 5.4.

Bảng 5.4: Các lớp lỗi EvoMaster v6.2.0 phát sinh trên biến thể có lỗ hổng

Ứng dụng	**Số lỗi**	**Các lớp lỗi phát sinh**
reports-api	17	109 (thiếu 401), 202 (sai schema 401), 305 (thông tin xác thực hợp lệ bị từ chối)
vampi-seeded	23	100 (HTTP 500), 109 (thiếu 401), 200 và 201 (sai schema)

Ca kiểm thử do v6.2.0 sinh ra đều thiếu bước hạ quyền và bước dùng lại token cấp trước khi hạ quyền, nên chưa có chuỗi nào thỏa mãn MR-A1 trên cả 2 ứng dụng này. Lớp lỗi gần MR-A1 nhất là 305, phát hiện thông tin xác thực hợp lệ bị từ chối tại một thời điểm. MR-A1 xét chiều ngược lại, khi token cấp trước lúc thu hồi quyền vẫn được chấp nhận. Bản v6.2.0 và phần mở rộng dùng bảng mã lỗi riêng, nên mã lỗi của v6.2.0 không đối chiếu trực tiếp được với mã của phần mở rộng.

Nhánh chính của EvoMaster upstream (commit `ab983ba338`, tháng 10 năm 2026) được chạy thêm với 20 seed trên biến thể có lỗ hổng của reports-api, qua một proxy ghi lại toàn bộ lưu lượng HTTP. Không lượt nào báo lỗi thuộc lớp MR-A1. Trong lưu lượng có 4 lời gọi dùng token cấp lúc tài khoản còn quyền quản trị, gửi sau khi tài khoản đã bị hạ quyền, và nhận mã 200. Không lời gọi nào trong số đó có source thành công bằng cùng token trước lần hạ quyền, không lời gọi nào bị gắn lỗi, và bộ ca kiểm thử xuất ra không giữ chuỗi nào như vậy.

Chương 6

Kết luận và hướng phát triển

Oracle 970 phát hiện được vi phạm MR-A1 trên cả bốn ứng dụng đã khảo sát, nhưng còn giới hạn ở khâu dựng bộ ba và khả năng mở rộng sang ứng dụng khác. Chương này tổng kết các đóng góp, nêu hạn chế và đề xuất hướng phát triển.

6.1 Tổng kết kết quả nghiên cứu

Khóa luận đã mở rộng EvoMaster bằng Oracle 970 để tự động kiểm tra lỗ hổng phân quyền có trạng thái thuộc quan hệ MR-A1. Phần mở rộng chạy trong pha kiểm thử bảo mật và không thay đổi thuật toán tìm kiếm. Các kết quả dưới đây được trình bày theo bốn đóng góp nêu ở Mục 1.4.

Thứ nhất, quan hệ MR-A1 được cài đặt thông qua điều kiện tiền đề và bộ ba thao tác `[source/transition/follow-up]`. Trên 98 lượt chạy, Oracle 970 phát hiện vi phạm ở 19/20 lượt với reports-api, 7/20 lượt với vampi-seeded, 5/5 lượt với pyLoad và 5/5 lượt với artifact-keeper. Công cụ không phát sinh báo động giả nào trong 48 lượt trên biến thể đã vá, trong đó 30 lượt có bộ ba hợp lệ. Mọi lượt bỏ sót đều không dựng được bộ ba hợp lệ để phán quyết.

Thứ hai, mô hình Naive Bayes tự động nhận diện thao tác chuyển trạng thái từ đặc tả OpenAPI. Trên 237 bản đặc tả không tham gia huấn luyện, 117 trong 153 thao tác chuyển trạng thái có điểm dương. Trong kiểm chứng chéo trên 47 bản đặc tả, bộ từ vựng đứng hạng 1 nhiều hơn, nhưng chỉ Naive Bayes luôn giữ thao tác đích trong 4 hạng đầu, với tối đa 3 ứng viên đứng trước. Trên pyLoad và artifact-keeper, công cụ tự chọn thao tác hạ quyền từ đặc tả mà không cần khai báo thủ công.

Thứ ba, bước đăng nhập lại loại các bộ ba mà token mới vẫn được chấp nhận, nhằm tránh phán quyết trên những thao tác chuyển trạng thái chưa xác nhận là hạ quyền. Trên các bộ ba hợp lệ đã được phán quyết, không ghi nhận trường hợp oracle bỏ sót vi phạm hay báo động giả.

Thứ tư, công cụ phát hiện hai lỗi thật không do tác giả viết. Trên pyLoad (CVE-2026-41133), Oracle 970 báo vi phạm ở 5/5 lượt trên biến thể có lỗ hổng và 0/5

lượt trên biến thể đã vá. Trên artifact-keeper, Oracle 970 báo vi phạm ở 5/5 lượt trên commit có lỗi và 0/3 lượt trên bản vá của nhà phát triển.

6.2 Hạn chế hiện tại của đề tài

Khóa luận chỉ kiểm chứng quan hệ MR-A1, tức trường hợp hạ vai trò; các quan hệ khác của cùng lớp lỗi được nêu ở Mục 6.3. Mã từ chối được cố định là 403 và chưa cấu hình được; bản vá của artifact-keeper trả 401 cho token cũ, nên công cụ không dựng được bộ ba nào trên bản vá này. pyLoad và artifact-keeper là ứng dụng dùng trong lúc phát triển bộ dựng, nên khóa luận không khẳng định tỷ lệ phát hiện trên ứng dụng chưa từng chạy. Các giới hạn còn lại, gồm quy mô mẫu nhỏ và phán quyết dựa trên mã trạng thái HTTP, được phân tích ở Mục 5.5.

6.3 Hướng phát triển trong tương lai

Để nhận diện thao tác chuyển trạng thái trong các đặc tả đặt tên không theo quy ước, có thể dùng mô hình ngôn ngữ lớn (Large Language Model, LLM) đề xuất ứng viên. Naive Bayes hiện chỉ dựa vào token đường dẫn, tên trường và phương thức. LLM có thể khai thác mô tả trong đặc tả OpenAPI và ngữ cảnh thao tác để đề xuất ứng viên, trong khi phép phán quyết giữ nguyên.

Phần mở rộng cũng có thể nhận thêm các tín hiệu từ chối ngoài mã 403, như mã 401 khi phiên bị vô hiệu hóa, để đánh giá được những bản vá như của artifact-keeper. Ngoài ra, bộ dựng có thể chủ động tạo tài nguyên và tài khoản cần thiết trước khi chạy kịch bản kiểm thử MR-A1, thay vì chỉ dựa vào kho lưu trữ. Cải tiến này nhắm vào 6 trong 14 lượt bỏ sót, nơi bộ dựng không thực hiện lần thử nào.

Ngoài MR-A1, nghiên cứu tiếp theo có thể xét các quan hệ khác của lớp phân quyền có trạng thái, như chuyển quyền sở hữu tài nguyên và quyền hết hạn theo thời gian. Mỗi quan hệ cần điều kiện tiền đề và cơ chế phán quyết riêng. Các hướng trên chưa được kiểm chứng trong phạm vi khóa luận.

Tài liệu tham khảo

[1] OWASP Foundation. OWASP API Security Top 10 2023. `https://owasp.org/API-Security/`, 2023.

[2] Andrea Arcuri. RESTful API Automated Test Case Generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 28(1):3:1–3:37, 2019.

[3] Ömür Şahin, Man Zhang, and Andrea Arcuri. Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks. *arXiv preprint arXiv:2604.00702*, 2026.

[4] MITRE. CWE-613: Insufficient Session Expiration. `https://cwe.mitre.org/data/definitions/613.html`, 2024.

[5] Andrea Arcuri. RESTful API Automated Test Case Generation. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, pages 9–20, 2017.

[6] Andrea Arcuri, Juan Pablo Galeotti, Bogdan Marculescu, and Man Zhang. EvoMaster: A Search-Based System Test Generation Tool. *Journal of Open Source Software*, 6(57):2153, 2021.

[7] Andrea Arcuri. Many Independent Objective (MIO) Algorithm for Test Suite Generation. In *Search Based Software Engineering*, pages 3–17. Springer, 2017.

[8] Andrea Arcuri. EvoMaster: Evolutionary Multi-context Automated System Test Generation. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, pages 394–397, 2018.

[9] Andrea Arcuri, Man Zhang, Susruthan Seran, Juan Pablo Galeotti, Amid Golmohammadi, Onur Duman, Agustina Aldasoro, and Hernan Ghianni. Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs. *Automated Software Engineering*, 32(1):4, 2025.

[10] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[11] Tsong Yueh Chen, Shing Chi Cheung, and Shiu Ming Yiu. Metamorphic Testing: A New Approach for Generating Next Test Cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998.

[12] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys (CSUR)*, 51(1):4:1–4:27, 2018.

[13] Phu X. Mai, Arda Goknil, Fabrizio Pastore, and Lionel C. Briand. SMRL: A Metamorphic Security Testing Tool for Web Systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, pages 9–12, 2020.

[14] BSR INCITS 359 Committee. Role Based Access Control (Draft). Technical report, InterNational Committee for Information Technology Standards (INCITS), April 2003.

[15] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. Proposed NIST Standard for Role-Based Access Control. *ACM Transactions on Information and System Security (TISSEC)*, 4(3):224–274, 2001.

[16] Phil Hunt, Kelly Grizzle, Erik Wahlstroem, and Chuck Mortimore. System for Cross-domain Identity Management: Core Schema. RFC 7643, RFC Editor, September 2015.

[17] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: Stateful REST API Fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 748–758, 2019.

[18] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. RestTestGen: Automated Black-Box Testing of RESTful APIs. In *Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 142–152, 2020.

[19] Andrea Arcuri, Ömür Şahin, and Man Zhang. Fuzzing for Detecting Access Policy Violations in REST APIs. In *Proceedings of the 36th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 130–141, 2025.

[20] Fengyu Liu, Yuan Zhang, Enhao Li, Wei Meng, Youkun Shi, Qianheng Wang, Chenlin Wang, Zihan Lin, and Min Yang. BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1320–1333, 2025.

[21] Omar Tsai, Jianing Li, Tsz Tung Cheung, Lejing Huang, Hao Zhu, Jianrui Xiao, Iman Sharafaldin, and Mohammad A. Tayebi. GraphQLer: Enhancing GraphQL Security with Context-Aware API Testing. *arXiv preprint arXiv:2504.13358v2*, 2026.

[22] Shehroz Khan, Abdullah Mughees, Gaadha Sudheerbabu, Tanwir Ahmad, and Dragos Truscan. Multi-Agent LLM-based Metamorphic Testing for REST APIs. *arXiv preprint arXiv:2605.28321*, 2026.

[23] I Putu Arya Dharmaadi, Mohannad Alhanahnah, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. BACFuzz: Exposing the Silence on Broken Access Control Vulnerabilities in Web Applications. *arXiv preprint arXiv:2507.15984*, 2025.

[24] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Semantics. RFC 9110, RFC Editor, June 2022.

[25] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, RFC Editor, May 2015.

[26] MITRE CVE Program. CVE-2026-41133: pyLoad Privilege Revocation Bypass. `https://cve.org/CVERecord?id=CVE-2026-41133`, 2026.

[27] artifact-keeper Contributors. fix(security): invalidate tokens on admin/role privilege change (Closes #1821). `https://github.com/artifact-keeper/artifact-keeper/commit/e64cc5c608df4afdd0b35feae5f140a9d809a8f3`, 2026.

[28] C. J. Clopper and E. S. Pearson. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika*, 26(4):404–413, 1934.

[29] Ronald A. Fisher. On the Interpretation of χ^2 from Contingency Tables, and the Calculation of P. *Journal of the Royal Statistical Society*, 85(1):87–94, 1922.

Phụ lục A

Danh mục endpoint của các ứng dụng mục tiêu

Bảng A.1 liệt kê các endpoint tham gia bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu mô tả ở Mục 4.2.

Bảng A.1: Các endpoint trong bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu

Endpoint	**Quyền**	**Vai trò trong kịch bản**
reports-api		
`POST /auth/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa vai trò
`PUT /reports/{id}`	Quản trị	Source và follow-up
`PUT /users/{u}/role`	Quản trị	Chuyển trạng thái
vampi-seeded		
`POST /users/v1/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa cờ quản trị
`PUT /books/v1/{t}/secret`	Quản trị	Source và follow-up
`PUT /users/v1/{u}/role`	Quản trị	Chuyển trạng thái
pyLoad		
`POST /login`	Mọi tài khoản	Đăng nhập, tạo cookie phiên
`POST /json/status`	Quản trị	Source và follow-up
`POST /api/set_user_permission`	Quản trị	Chuyển trạng thái
artifact-keeper		
`POST /api/v1/auth/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa cờ `is_admin`
`POST /api/v1/admin/analytics/snapshot`	Quản trị	Source và follow-up
`PATCH /api/v1/users/{id}`	Quản trị	Chuyển trạng thái

Trên pyLoad, nhóm đường dẫn `/json/*` từ chối bằng mã 403, còn nhóm `/api/*` trả mã 401. Vì MR-A1 chỉ coi 403 là lời từ chối, thao tác source hợp lệ chỉ đến từ nhóm

`/json/*`. Phép kiểm tra HTTP thủ công trên pyLoad dùng source `POST /json/generate_apikey` và thao tác chuyển trạng thái `POST /json/update_users`.